

Data-Link Layer: Error Control

Dr. B. R. Chandavarkar,

Dept. of CSE,

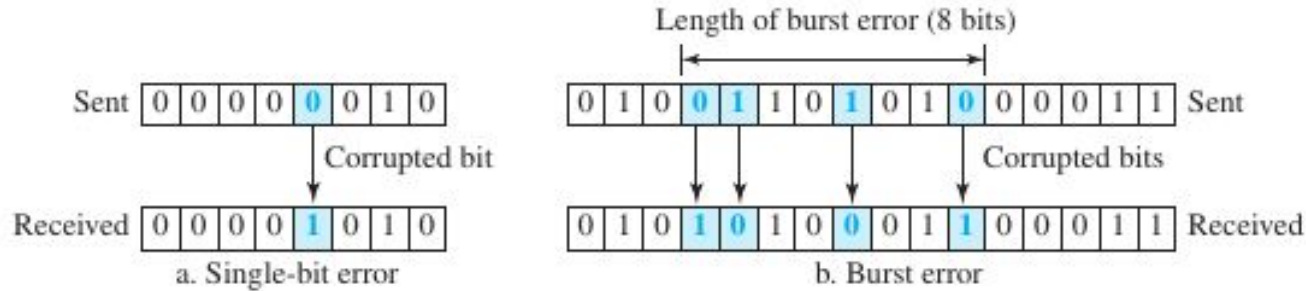
NITK, Surathkal

brc@nitk.edu.in

Contents:

- Type of errors
- Redundancy
- Coding
- Block coding
- Hamming distance
- Linear block code
- Cyclic block code
- Convolution code

Types of error:



- A **burst error** is more likely to occur than a **single-bit error** because the duration of the noise signal is normally longer than the duration of 1 bit, which means that when noise affects data, it affects a set of bits.
- The number of bits affected depends on the data rate and duration of noise.
- For example, if we are sending data at 1 kbps, a noise of 1/100 second can affect 10 bits; if we are sending data at 1 Mbps, the same noise can affect 10,000 bits.

Redundancy

- The central concept in detecting or correcting errors is redundancy.
- To be able to detect or correct errors, we need to send some extra bits with our data.
- These redundant bits are added by the sender and removed by the receiver.
- Their presence allows the receiver to detect or correct corrupted bits.
- Redundancy is achieved through various coding schemes.
- The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits.
- The receiver checks the relationships between the two sets of bits to detect errors.
- The ratio of redundant bits to data bits and the robustness of the process are important factors in any coding scheme.

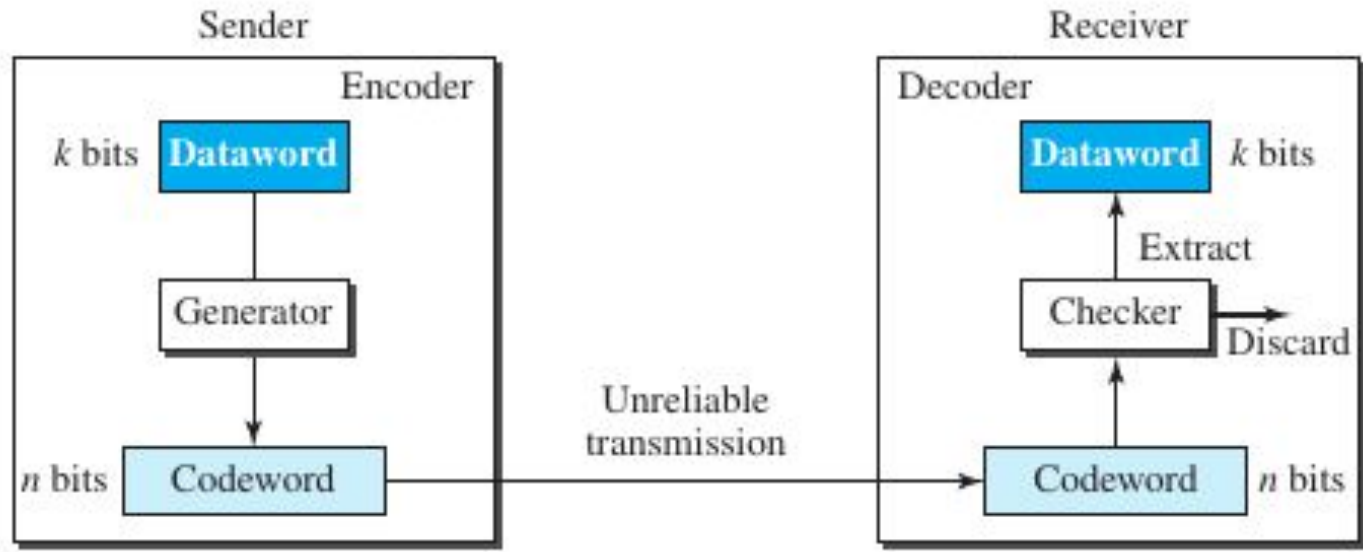
Coding

- The coding schemes can be block coding or convolution coding.

Block coding

- In block coding, message is divided into blocks, each of k bits, called **datawords**.
- Add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called **codewords**.
- Since $n > k$, the number of possible codewords is larger than the number of possible datawords.
- The block coding process is one-to-one; the same dataword is always encoded as the same codeword.
- This means that we have $2^n - 2^k$ codewords that are not used. We call these codewords invalid or illegal.

Process of error detection in block coding



Example:

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
00	000	10	101
01	011	11	110

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

- The receiver receives 011.
 - It is a valid codeword. The receiver extracts the dataword 01 from it.
- The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted).
 - This is not a valid codeword and is discarded.
- The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted).
 - This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

Hamming Distance

- One of the central concepts in coding for error control is the idea of the Hamming distance.
- The Hamming distance between two words (of the same size) is the number of differences between the corresponding bits.
- We show the Hamming distance between two words x and y as $d(x, y)$.
- For example, if the codeword 00000 is sent and 01101 is received, 3 bits are in error and the Hamming distance between the two is $d(00000, 01101) = 3$.
- In other words, if the Hamming distance between the sent and the received codeword is not zero, the codeword has been corrupted during transmission.

Minimum Hamming Distance for Error Detection

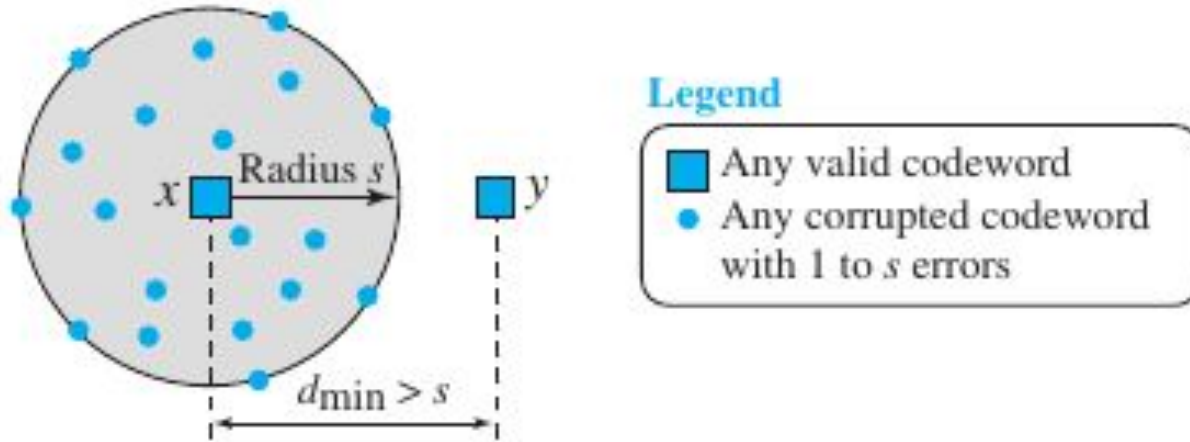
- In a set of codewords, the minimum Hamming distance is the smallest Hamming distance between all possible pairs of codewords.
- Example:

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
00	000	10	101
01	011	11	110

$d_{\min} = 2$. ($d(000, 011) = 2$, $d(000, 101) = 2$, $d(000, 110) = 2$, $d(011, 101) = 2$, $d(011, 110) = 2$, and $d(101, 110) = 2$).

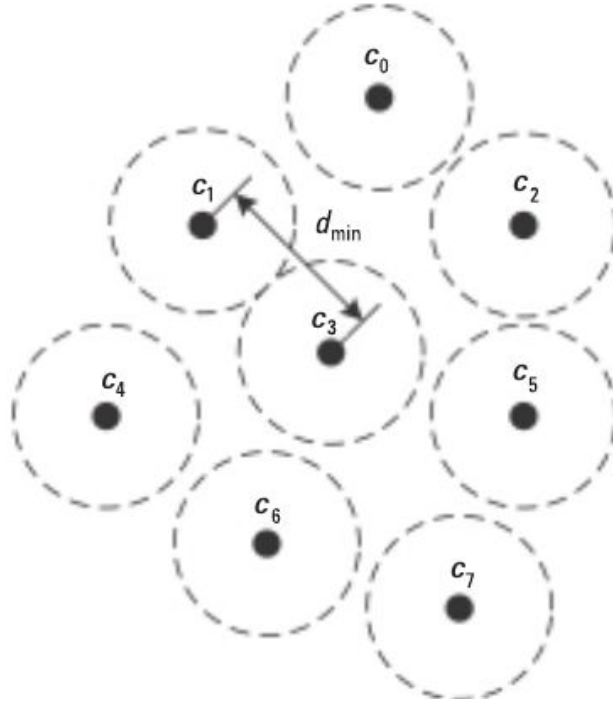
- The minimum Hamming distance to detect 's' bit errors:
 - If s errors occur during transmission, the Hamming distance between the sent codeword and received codeword is s.
 - If our system is to detect up to s errors, the minimum distance between the valid codes must be $(s + 1)$, so that the received codeword does not match a valid codeword.
 - In other words, if the minimum distance between all valid codewords is $(s + 1)$, the received codeword cannot be erroneously mistaken for another codeword → The error will be detected.

A geometric concept of $d_{\min}$ in error detection



$d_{\min} = s + 1 \rightarrow$ To detect the error, other valid codeword should be at a distance of $s + 1$.

A geometric concept of $d_{\min}$ in error correction



For correcting 't' errors, $d_{\min} = 2*t + 1 \rightarrow$ To correct the error, other valid codeword should be at a distance of $2*t + 1$.

Block coding:

- A block code is specified by a set of two parameters (n, k) , where n is the length of the codeword and k is the message word length.
- The rate of the code is k/n .
- Each message word is associated with one and only one codeword.
- The total number of codewords in a code equals that of message words, 2^k .
- **Two type:** Linear block code and cyclic block code

Codewords of a (7,4) Block Code

Message Word (LSB ... MSB)	Codeword (LSB ... MSB)
$m_0 = (0000)$	$c_0 = (0000000)$
$m_1 = (1000)$	$c_1 = (1101000)$
$m_2 = (0100)$	$c_2 = (0110100)$
$m_3 = (1100)$	$c_3 = (1011100)$
$m_4 = (0010)$	$c_4 = (1110010)$
$m_5 = (1010)$	$c_5 = (0011010)$
$m_6 = (0110)$	$c_6 = (1000110)$
$m_7 = (1110)$	$c_7 = (0101110)$
$m_8 = (0001)$	$c_8 = (1010001)$
$m_9 = (1001)$	$c_9 = (0111001)$
$m_{10} = (0101)$	$c_{10} = (1100101)$
$m_{11} = (1101)$	$c_{11} = (0001101)$
$m_{12} = (0011)$	$c_{12} = (0100011)$
$m_{13} = (1011)$	$c_{13} = (1001011)$
$m_{14} = (0111)$	$c_{14} = (0010111)$
$m_{15} = (1111)$	$c_{15} = (1111111)$

Note: LSB = least significant bit; MSB = most significant bit.

A. The **linear block codes** have the following two important properties:

1. The sum of any two codewords in C is another codeword in C :

$$\mathbf{c}_i + \mathbf{c}_j = \mathbf{c}_k \quad (3.1)$$

where $\mathbf{c}_i$, $\mathbf{c}_j$ and $\mathbf{c}_k \in C$.

2. There exists a set of k codewords in C which are *linearly independent* such that every codeword in C is a linear combination of the k codewords:

$$\mathbf{c} = m_0 \mathbf{g}_0 + m_1 \mathbf{g}_1 + \cdots + m_{k-1} \mathbf{g}_{k-1} \quad (3.2)$$

where $\mathbf{g}_0, \mathbf{g}_1, \mathbf{g}_2, \cdots \mathbf{g}_{k-1}$ are the k linearly independent codewords, and $m_0, m_1, m_2, \cdots m_{k-1}$ are some scalars. By linearly independent we mean that $m_0 \mathbf{g}_0 + m_1 \mathbf{g}_1 + \cdots + m_{k-1} \mathbf{g}_{k-1} \neq 0$ unless $m_0, m_1, \cdots, m_{k-1} = 0$.

Example:

Property-1:

$$c_6 = (1000110) \text{ and } c_8 = (1010001) \rightarrow c_{14} = c_6 \oplus c_8$$

Property-2:

$$c_1 = (1010001), c_2 = (1110010), c_4 = (0110100), \text{ and } c_8 = (1101000).$$

$$\rightarrow c_9 = 1 \times c_1 \oplus 0 \times c_2 \oplus 0 \times c_4 \oplus 1 \times c_8$$

$$\rightarrow c_{13} = 1 \times c_1 \oplus 0 \times c_2 \oplus 1 \times c_4 \oplus 1 \times c_8$$

$$\mathbf{c} = m_0 \mathbf{g}_0 + m_1 \mathbf{g}_1 + \cdots + m_{k-1} \mathbf{g}_{k-1}$$

$$\mathbf{c} = (m_0 m_1 m_2 \cdots m_{k-1}) \cdot \begin{bmatrix} \mathbf{g}_0 \\ \mathbf{g}_1 \\ \vdots \\ \mathbf{g}_{k-1} \end{bmatrix}$$

$$= \underbrace{(m_0 m_1 m_2 \cdots m_{k-1})}_{\mathbf{m}} \cdot \underbrace{\begin{bmatrix} g_{0,0} & g_{0,1} & \cdots & g_{0,n-1} \\ g_{1,0} & g_{1,1} & \cdots & g_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ g_{k-1,0} & g_{k-1,1} & \cdots & g_{k-1,n-1} \end{bmatrix}}_{\mathbf{G}}$$

$$= \mathbf{m} \cdot \mathbf{G}$$

- where $\mathbf{m} = (m_0 \ m_1 \ m_2 \ \dots \ m_{k-1})$ is the message word, and
- the $k \times n$ matrix $\mathbf{G}$ is called the **generator matrix** of the code $\mathbf{C}$.
- A linear block code is completely determined by its generator matrix.

For systematic codes, the generator matrix G consists of a $k \times (n - k)$ matrix P , which produces the parity-check bits of the code, and a $k \times k$ identity matrix I_k , side by side:

$$\begin{aligned}
 \mathbf{G} = & \left[\begin{array}{cccc|ccccc} p_{0,0} & p_{0,1} & \cdots & p_{0,n-k-1} & 1 & 0 & \cdots & 0 & 0 \\ p_{1,0} & p_{1,1} & \cdots & p_{1,n-k-1} & 0 & 1 & \cdots & 0 & 0 \\ \vdots & & & & \vdots & & \ddots & & \vdots \\ p_{k-1,0} & p_{k-1,1} & \cdots & p_{k-1,n-k-1} & 0 & 0 & \cdots & 0 & 1 \end{array} \right] = [\mathbf{P} \mid \mathbf{I}_k] \\
 & \qquad \qquad \qquad \widetilde{\mathbf{P}} \qquad \qquad \qquad \qquad \qquad \qquad \qquad \widetilde{\mathbf{I}}_k
 \end{aligned}$$

Message Word (LSB ... MSB)	Codeword (LSB ... MSB)
$m_0 = (0000)$	$c_0 = (0000000)$
$m_1 = (1000)$	$c_1 = (1101000)$
$m_2 = (0100)$	$c_2 = (0110100)$
$m_3 = (1100)$	$c_3 = (1011100)$
$m_4 = (0010)$	$c_4 = (1110010)$
$m_5 = (1010)$	$c_5 = (0011010)$
$m_6 = (0110)$	$c_6 = (1000110)$
$m_7 = (1110)$	$c_7 = (0101110)$
$m_8 = (0001)$	$c_8 = (1010001)$
$m_9 = (1001)$	$c_9 = (0111001)$
$m_{10} = (0101)$	$c_{10} = (1100101)$
$m_{11} = (1101)$	$c_{11} = (0001101)$
$m_{12} = (0011)$	$c_{12} = (0100011)$
$m_{13} = (1011)$	$c_{13} = (1001011)$
$m_{14} = (0111)$	$c_{14} = (0010111)$
$m_{15} = (1111)$	$c_{15} = (1111111)$

Note: LSB = least significant bit; MSB = most significant bit.

The generator matrix for the (7,4) systematic code example is:

$$G = \begin{bmatrix} g_0 \\ g_1 \\ g_2 \\ g_3 \end{bmatrix} = \begin{bmatrix} c_8 \\ c_4 \\ c_2 \\ c_1 \end{bmatrix} = \left[\begin{array}{ccc|ccc} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{array} \right]$$

‘G’ is used for encoding (dataword → codeword conversion).

- A closely associated matrix with 'G' is the parity-check matrix (H).
- The relation between G and H is $G \cdot H^T = 0$, where, H^T is the transpose of H.
- $H_{r \times n} = [I_{r \times r} \mid P_{r \times k}^T]$, where, $I \rightarrow$ Identity matrix and is P^T the transpose of P.

- **Summary:**

- $m \rightarrow$ dataword of length 'k' bits.
- $c \rightarrow$ codeword of length 'n' bits.
- $r \rightarrow$ redundant bits of $n - k$ length.
- $(n, k) \rightarrow$ Block code
- $G_{k \times n} = [P_{k \times r} \mid I_{k \times k}] \rightarrow$ Generator matrix used for encoding
- $H_{r \times n} = [I_{r \times r} \mid P_{r \times k}^T] \rightarrow$ Parity-check matrix used for decoding
- $c = m \cdot G$
- $s = c \cdot H^T \rightarrow$ Syndrome

k → Dataword, **n** → Codeword, and **r** → Parity/Redundant bit relation:

How to decide 'r' for 'k'?

- $(k + r + 1) \leq 2^r$ Where, $n = k + r$
- $2^r - r - 1 \geq k$

Hamming Codes with Different (n, k)

n	k	$g(X)$
7	4	$1 + X + X^3$
15	11	$1 + X + X^4$
31	26	$1 + X^2 + X^5$
63	57	$1 + X + X^6$
127	120	$1 + X^3 + X^7$
255	247	$1 + X^2 + X^3 + X^4 + X^8$
511	502	$1 + X^4 + X^9$
1,023	1,013	$1 + X^3 + X^{10}$
2,047	2,036	$1 + X^2 + X^{11}$
4,095	4,083	$1 + X + X^4 + X^6 + X^{12}$

[Factors of polynomial](#)

Syndrome and bit with error relation:

- Let, dataword = $m_3 m_2 m_1 m_0$ and codeword $(c) = c_6 c_5 c_4 c_3 c_2 c_1 c_0$ in $(7, 4)$.
- Generator matrix $(G) = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$

and

$$\text{Parity check matrix } (H) = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

- So, in the calculation of $c = m \cdot G$, $c_6 = m_3 \oplus m_2 \oplus m_1$, $c_5 = m_3 \oplus m_1 \oplus m_0$, $c_4 = m_2 \oplus m_1 \oplus m_0$, $c_3 = m_3$, $c_2 = m_2$, $c_1 = m_1$, and $c_0 = m_0$.

Syndrome and bit with error relation:

- Let the received codeword $(c') = c_6' c_5' c_4' c_3' c_2' c_1' c_0'$.
- So, in the calculation of r-bit syndrome $(s_2 s_1 s_0) = c' \cdot H^T$,
 $s_2 = c_6' \oplus c_3' \oplus c_2' \oplus c_1'$
 $s_1 = c_5' \oplus c_3' \oplus c_1' \oplus c_0'$ and
 $s_0 = c_4' \oplus c_2' \oplus c_1' \oplus c_0'$
- Accordingly, syndrome $\rightarrow$ bit with error is, $000 \rightarrow$ No error, $001 \rightarrow c_4'$, $010 \rightarrow c_5'$, $011 \rightarrow c_1'$, $100 \rightarrow c_6'$, $101 \rightarrow c_2'$, $110 \rightarrow c_3'$, and $111 \rightarrow c_1'$.

Example:

- n (codeword length) = 7, k (dataword length) = 4, r (redundant bits) = $n - k = 3$ and the block code (7, 4) with $d_{\min} = 3$.
- $G_{4 \times 7} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0; \\ 1 & 0 & 1 & 0 & 1 & 0 & 0; \\ 1 & 1 & 1 & 0 & 0 & 1 & 0; \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$
- $H_{3 \times 7} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0; \\ 0 & 1 & 0 & 1 & 0 & 1 & 1; \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$
- If $m = [1 \ 0 \ 1 \ 1]$ then $c = m \cdot G = [0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1]$
- $c \cdot H^T = [0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1] \cdot H = [0 \ 0 \ 0] \rightarrow$ No error

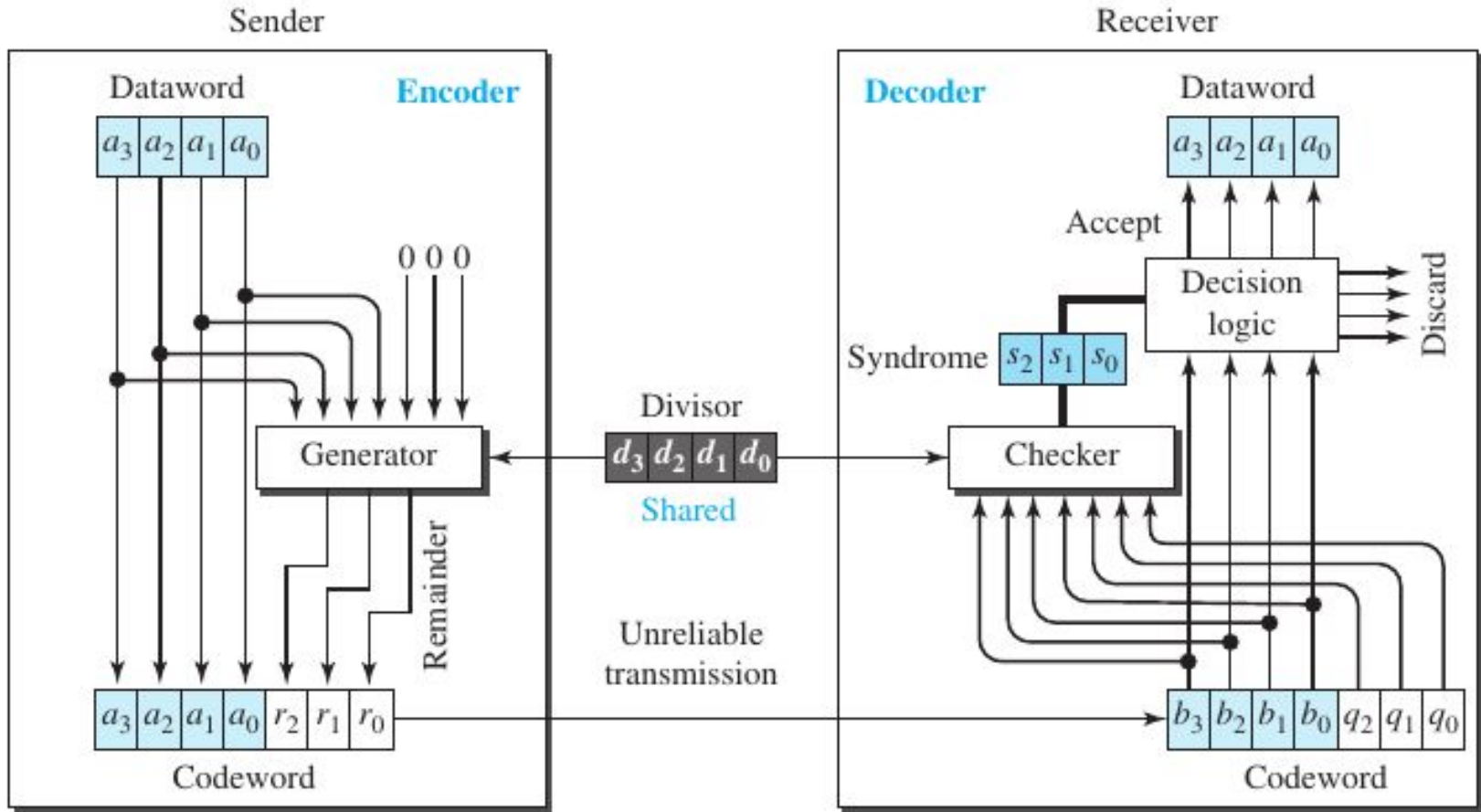
Cyclic codes

- Cyclic codes are special linear block codes with one extra property.
- In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.

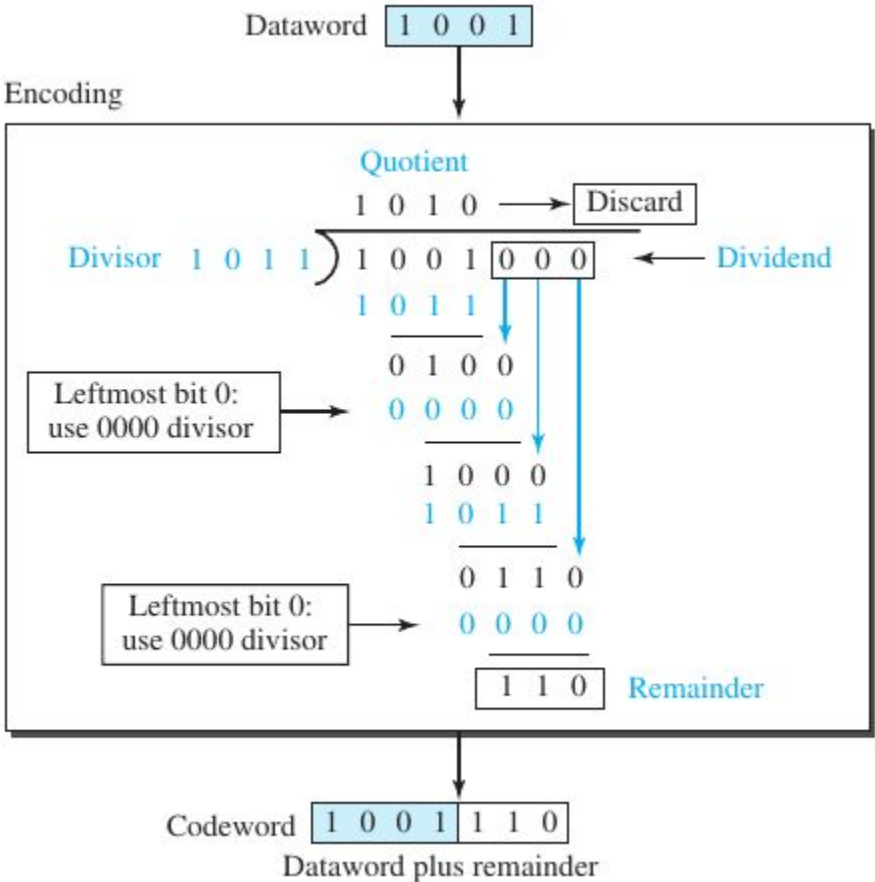
Table 10.3 A CRC code with $C(7, 4)$

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

CRC Encoder and Decoder



Example: CRC Encoding



Note:
Multiply: AND
Subtract: XOR

Example: CRC Decoding

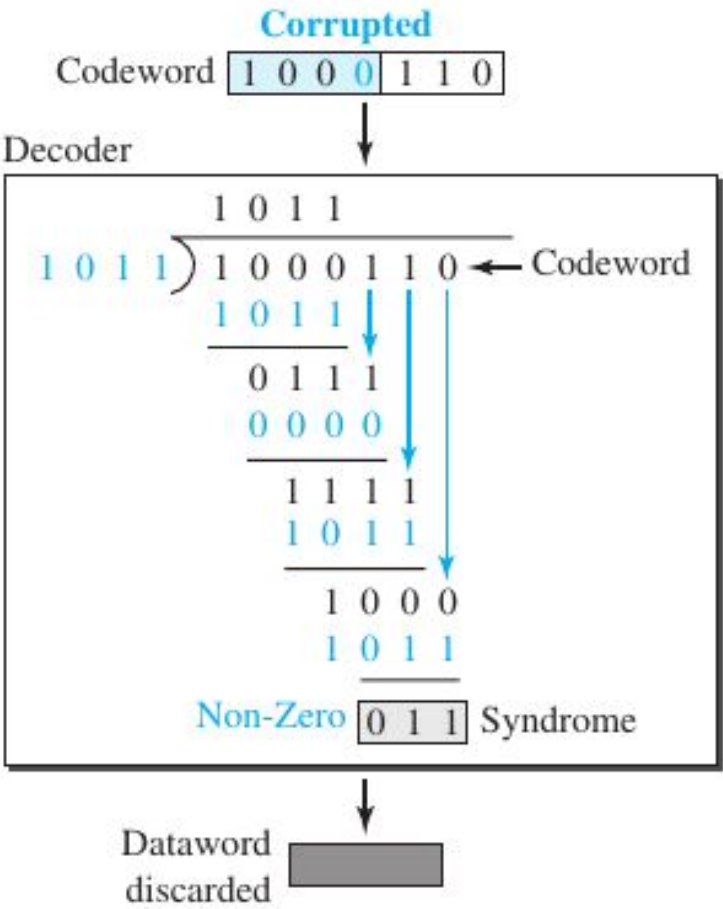
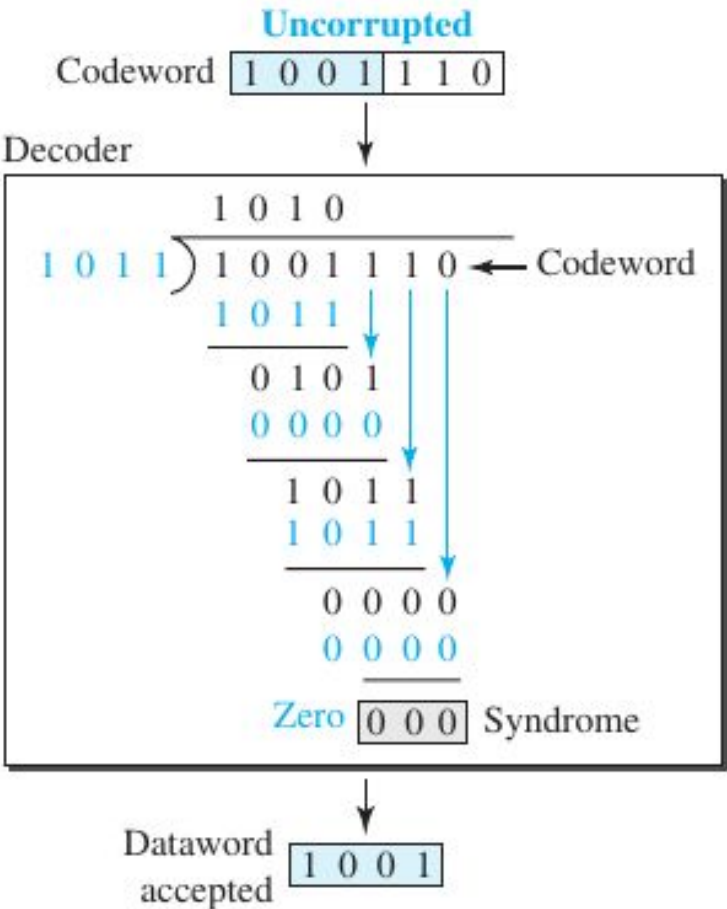


Table 10.4 *Standard polynomials*

<i>Name</i>	<i>Polynomial</i>	<i>Used in</i>
CRC-8	$x^8 + x^2 + x + 1$ 100000111	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$ 11000110101	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$ 10001000000100001	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ 100000100110000010001110110110111	LANs

- Cyclic codes form a subclass of linear block codes.
- Indeed, many of the important linear block codes discovered to date are either cyclic codes or closely related to cyclic codes.
- An advantage of cyclic codes over most other types of codes is that they are easy to encode.
- Cyclic code property:
 - **Linearity Property:** The sum of any two codewords in the code is also a codeword.
 - **Cyclic Property:** Any cyclic shift of a codeword in the code is also a codeword.

- To restate Property 2 in mathematical terms, let the n -tuple $c_0, c_1, c_2, \dots, c_{n-1}$ denote a codeword of an (n, k) linear block code. The code is a cyclic code if the n -tuples

$$\begin{array}{c} (c_{n-1}, c_0, \dots, c_{n-2}) \\ (c_{n-2}, c_{n-1}, \dots, c_{n-3}) \\ \vdots \\ (c_1, c_2, \dots, c_{n-1}, c_0) \end{array}$$

are all codewords in the code.

- To develop the algebraic properties of cyclic codes, we use the elements $c_0, c_1, \dots, c_{n-1}$ of a codeword to define the code polynomial

$$c(X) = c_0 + c_1X^1 + c_2X^2 + \dots + c_{n-1}X^{n-1}$$

$$c^{(i)}(X) = c_{n-i} + c_{n-i+1}X^1 + \dots + c_{n-1}X^{i-1} + c_0X^i + \dots + c_{n-i-1}X^{n-1}$$

- Each power of X in the polynomial $c(X)$ represents a one-bit shift in time.
- Hence, multiplication of the polynomial $c(X)$ by X may be viewed as a shift to the right.
- For cyclic shift multiply $c(X)$ by X^i

$$X^i c(X) = c_0X^i + c_1X^{i+1} + \dots + c_{n-i-1}X^{n-1} + \dots + c_{n-1}X^{n+i-1}$$

$$X^i \mathbf{c}(X) = c_0 X^i + c_1 X^{i+1} + \cdots + c_{n-i-1} X^{n-1} + \cdots + c_{n-1} X^{n+i-1}$$

The above equation can be rewritten as,

$$\begin{aligned} X^i c(X) = & c_{n-i} + c_{n-i+1} X^1 + \dots + c_{n-1} X^{i-1} + c_0 X^i + \dots + \\ & c_{n-i-1} X^{n-1} + c_{n-i}(X^n - 1) + c_{n-i+1} X(X^n - 1) + \dots + \\ & c_{n-1} X^{i-1}(X^n - 1) \end{aligned}$$

$$X^i c(X) = q(X)(X^n - 1) + c^{(i)}(X)$$

Where,

$$\mathbf{q}(X) = c_{n-i} + c_{n-i+1} X + \cdots + c_{n-1} X^{i-1}$$

$\mathbf{c}^{(i)}(X)$ is the remainder that results from dividing $X^i \mathbf{c}(X)$ by $(X^n + 1)$

Example: For instance, the code polynomial of $c = [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1]$ is $c(X) = 1 + X^2 + X^6$.
 The code polynomial of $c^{(2)} = [0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0]$ is obtained as:

$$\begin{array}{r}
 X^7 - 1 \overline{) \begin{array}{c} X \\ X^8 + X^4 + X^2 \end{array}} \\
 \underline{\begin{array}{c} X^8 + X \end{array}} \\
 X^4 + X^2 + X \leftarrow \text{code polynomial of } c^{(2)}
 \end{array}$$

- The polynomial $X^n - 1$ and its factors play a major role in the generation of cyclic codes. Let $g(X)$ be a polynomial of degree $n - k$ that is a factor of $X^n - 1$; as such, $g(X)$ is the polynomial of least degree in the code with g_0 and $g_{n-k} = 1$.

$$g(X) = g_0 + g_1X + g_2X^2 + \cdots + g_{n-k}X^{n-k}$$

- **Example:**
 - The polynomial $X^7 + 1$ can be factored as, $(1 + X + X^3)(1 + X + X^2 + X^4)$ or $(1 + x)(1 + X + X^3)(1 + X^2 + X^3)$, $g(X) = (1 + X + X^3)$ or $(1 + X^2 + X^3)$.
- The polynomial $g(X)$ is called the **generator polynomial** of a cyclic code.
- A cyclic code is uniquely determined by the generator polynomial $g(X)$ in that each code polynomial in the code can be expressed in the form of a polynomial product as follows: $c(X) = m(X) \cdot g(X)$, where, $m(X) \rightarrow$ is a polynomial in X with degree $k - 1$.

Example: Suppose $m = [1 \ 0 \ 1 \ 0]$ be the message to be encoded using the generator polynomial $g(X) = 1 + X + X^3$.

Ans:

$c(X) = m(X) * g(X) = (1 + X^2) (1 + X + X^3) = 1 + X + X^2 + X^5 = [1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0] \rightarrow$ but, not in systematic form.

To put in systematic form:

Step-1: Multiply $m(X)$ by $X^{r = n - k}$

Step-2: Divide $m(X) * X^{n - k}$ by $g(X)$.

Step-3: Remainder $b(X) + m(X) * X^{n-k} \rightarrow c(X)$ in systematic form

Example: Consider (7, 4) cyclic code with $g(X) = 1 + X + X^3$. Encode $m(X) = [1\ 0\ 0\ 1] = 1 + X^3$

Ans:

Step-1: $X^3 * m(X) = X^3 + X^6$

Step-2: Divide $X^3 + X^6$ by $g(X) \rightarrow b(X) = X^2 + X$

$$\begin{array}{r}
 \overline{X^3 + X} \text{ (quotient)} \\
 X^3 + X + 1 \overline{)X^6} \\
 \underline{X^6} \\
 X^4 \\
 \underline{X^4} \\
 X^2 + X \text{ (remainder),}
 \end{array}$$

Step-3: $c(X) = b(X) + X^3 * m(X) = X + X^2 + X^3 + X^6 = [0\ 1\ 1\ 1\ 0\ 0\ 1]$

- Similar to the parity-check matrix, a parity-check polynomial is associated with cyclic codes:

$$h(X) = h_0 + h_1X + h_2X^2 + \dots + h_kX^k$$

where $h_0, h_k = 1$.

- Like the generator polynomial $g(X)$, the parity-check polynomial $h(X)$ also determines a code.
- The polynomials $h(X)$ and $g(X)$ are related to each other as follows:

$$g(X)h(X) = X^n - 1 \text{ and } h(X) = \frac{X^n - 1}{g(X)}$$

- Further,

$$\begin{aligned} c(X)h(X) \bmod (X^n - 1) &= m(X)g(X)h(X) \bmod (X^n - 1) \\ &= m(X)(X^n - 1) \bmod (X^n - 1) \\ &= 0 \end{aligned}$$

Decoding:

$$s(X) = c'(X) / g(X)$$

Example: In systematic code $c(X) = \text{redundant bit} + \text{data word}$

If data word = [1 0 1 1] and $g(X) = 1 + X + X^3$, then $c(X) = [1\ 0\ 0\ 1\ 0\ 1\ 1]$

Error code $\rightarrow$ Error polynomial pair is,

$[1\ 0\ 0\ 0\ 0\ 0\ 0] \rightarrow X^0$, $[0\ 1\ 0\ 0\ 0\ 0\ 0] \rightarrow X^1$, $[0\ 0\ 1\ 0\ 0\ 0\ 0] \rightarrow X^2$, $[0\ 0\ 0\ 1\ 0\ 0\ 0] \rightarrow X^3/g(X) = X + 1$, $[0\ 0\ 0\ 0\ 1\ 0\ 0] = X^4/g(X) = X + X^2$, $[0\ 0\ 0\ 0\ 0\ 1\ 0] = X^5/g(X) = X^2$, $[0\ 0\ 0\ 0\ 0\ 0\ 1] = X^6/g(x) = X^2 + 1$.

If $c'(X) = [1\ 0\ 0\ 1\ 0\ 1\ 0]$, then $s(X) = X^2 + 1 \rightarrow$ Error polynomial is X^6 .

$g(x) \longleftrightarrow \mathbf{G}$ relation:

- $g(x) = 1 + x + x^3$, $xg(x) = x + x^2 + x^4$, $x^2g(x) = x^2 + x^3 + x^5$, $x^3g(x) = x^3 + x^4 + x^6$ resulting in
- $G' = [g(x); xg(x); x^2g(x), x^3g(x)]$
 $= [\text{row-1: } 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0; \text{row-2: } 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0; \text{row-3: } 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0; \text{row-4: } 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1].$
- To convert G' to G , $\text{row-3} \rightarrow \text{row-1} \oplus \text{row-3}$ and $\text{row-4} \rightarrow \text{row-1} \oplus \text{row-2} \oplus \text{row-4}$, resulting $G = [1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0; 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0; 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0; 0 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1]$

$h(x) \longleftrightarrow \mathbf{H}$ relation:

- $h(x) = 1 + x + x^2 + x^4$, calculate the reciprocal of $h(x) \rightarrow x^4h(x^{-1}) = H(x) = 1 + x^2 + x^3 + x^4$
- **Example:** The reciprocal of $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ is $x^np(x^{-1}) = a_n + a_{n-1}x + a_{n-2}x^2 + \dots + a_0x^n$
- $H' = [H(x); xH(x); x^2H(x)] = [1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0; 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0; 0 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1]$

Hamming Code:

For, (7, 4) Hamming code, the code word is,

Position	7	6	5	4	3	2	1
Codeword	D4	D3	D2	P3	D1	P2	P1

Where, dataword $\rightarrow$ D4 D3 D2 D1

P1 (2^0 as 1) $\rightarrow$ 1, 3, 5, 7; P2 (2^1 as 1) $\rightarrow$ 2, 3, 6, 7; P3 (2^2 as 1) $\rightarrow$ 4, 5, 6, 7

Example: Dataword: 1 0 1 1; Codeword: 1 0 1 0 1 0 1

Bose-Chaudhuri-Hocquenghem (BCH)

- A widely used class of multi-bit error correcting codes
- An (n, k) BCH code with Hamming distance $D = 5$ is generally labeled BCH($n, k, 5$).
- **Summary** (Example):
 - $GF(2^3) \rightarrow \{000, 001, 010, 011, 100, 101, 110, 111\} \rightarrow \{0, 1, X, X + 1, X^2, X^2 + 1, X^2 + X, X^2 + X + 1\} \rightarrow \{0, X^2, X, X + X^2, 1 + X^2, 1 + X, 1 + X + X^2\}$
 - **Irreducible polynomial** (Prime polynomial): Any polynomial that has no factors other than 1 and itself. Factors for polynomial in GF(2): [Click here](#)
 - **Primitive polynomial**: Any irreducible polynomial $P(X)$ of degree N is primitive if $P(X)$ is a factor of $X^M + 1$ for $M = 2^N - 1$ and no smaller M . [Click here](#)
 - Example-1: $P(X) = X^3 + X + 1$ with $M = 2^3 - 1 = 7$ is a factor of $X^7 + 1$.
 - Example-2: $P(X) = X^4 + X^3 + X^2 + X + 1$ with $M = 2^4 - 1 = 15$ is a factor of $X^{15} + 1$ and also of $X^5 + 1$ ($= (x+1)(x^4+x^3+x^2+x+1)$) $\rightarrow$ So, $P(X)$ is Not a primitive factor.

- Primitive root:** The non-zero elements of the Galois field $GF(2^3)$ may be generated (not in order) as the set $\{a^0, a^1, a^2, \dots, a^6\}$ modulus a primitive polynomial $P(x)$ where a is called a primitive root of the field.

$GF(8)$ defined as $\{a^n\}$ with $P(x) = x^3 + x + 1$

$a^0 = 1$	$= 001 = 1$	$= a^7 = a^{14} = a^{21} = a^{28} \dots$
$a^1 = a$	$= 010 = 2$	$= a^8 = a^{15} = a^{22} = a^{29} \dots$
$a^2 = a^2$	$= 100 = 4$	$= a^9 = a^{16} = a^{23} = a^{30} \dots$
$a^3 = a+1$	$= 011 = 3$	$= a^{10} = a^{17} = a^{24} = a^{31} \dots$
$a^4 = a^2+a$	$= 110 = 6$	$= a^{11} = a^{18} = a^{25} = a^{32} \dots$
$a^5 = a^2+a+1$	$= 111 = 7$	$= a^{12} = a^{19} = a^{26} = a^{33} \dots$
$a^6 = a^2+1$	$= 101 = 5$	$= a^{13} = a^{20} = a^{27} = a^{34} \dots$

The i -th power of each of the elements a^N can be described as $a^{N \cdot i}$. From the values shown in the previous slide (GF(8)), these powers can be summarized as the octal values shown here:

i	a^{6i}	a^{5i}	a^{4i}	a^{3i}	a^{2i}	a^{1i}	a^{0i}
1	5	7	6	3	4	2	1
2	7	3	2	5	6	4	1
3	6	2	7	4	5	3	1
4	3	5	4	7	2	6	1
5	4	6	5	2	3	7	1
6	2	4	3	6	7	5	1
7	1	1	1	1	1	1	1
	A	B	C	D	E	F	G

Columns are labeled A..G for reference.

BCH - Single bit error correction:

- A (3 x 7) check matrix **H** for single-bit error correction is defined from the previous slide table (with i = 1) as shown below. In practice any of the rows (1 to 6) could be used.

$$H = \begin{matrix} & A & B & C & D & E & F & G \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & 5 & 7 & 6 & 3 & 4 & 2 & 1 \end{matrix}$$

$$H = \begin{matrix} & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{matrix}$$

- Syndrome

Table 1		$E(x)=0$	$E(x)=x^0$	$E(x)=x^1$	$E(x)=x^2$	$E(x)=x^3$	$E(x)=x^4$	$E(x)=x^5$	$E(x)=x^6$
Error		00000000	00000001	00000010	00001000	00010000	00100000	01000000	10000000
LABEL			G	F	E	D	C	B	A
Syndrome		0	1	2	4	3	6	7	5

Example:

- $m = [1011] = x^3 + x + 1$, $g(x) = x^3 + x + 1$. The codeword (c) in systematic form after following 3 steps becomes $[1011000]$
- The check matrix $H = [5 \ 7 \ 6 \ 3 \ 4 \ 2 \ 1]$
- Syndrome = $c' \cdot H^T$
- If the received $c' = [1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1]$ results in syndrome = $[0 \ 0 \ 1] = x^0$
- If the received $c' = [1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0]$ results in syndrome = $[1 \ 1 \ 1] = x^5$
- If the received $c' = [0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 0]$ results in syndrome = $[1 \ 0 \ 1] = x^6$

BCH - Double bit error correction:

- A code with this property can be constructed from the previous single bit error correcting matrix by extending the matrix (with another single bit error correcting matrix) to give a double bit error correcting check matrix.
- The challenge is always to determine how to extend this matrix to assure that all one- and two-bit errors will give a unique syndrome.
- Note that in every row ($i = 1, 2, \dots, 7$) the different column ordering gives a different syndrome for any specific error. The combination of two rows from the table is key to this next step.
- This additional row will add three rows to the binary matrix H .

- The (6 x 7) check matrix **H** for double-bit error correction is defined with $i = (1, 3)$ shown below.

$$\mathbf{H} = \begin{matrix} & \mathbf{A} & \mathbf{B} & \mathbf{C} & \mathbf{D} & \mathbf{E} & \mathbf{F} & \mathbf{G} \\ \mathbf{5} & \mathbf{7} & \mathbf{6} & \mathbf{3} & \mathbf{4} & \mathbf{2} & \mathbf{1} \\ \mathbf{6} & \mathbf{2} & \mathbf{7} & \mathbf{4} & \mathbf{5} & \mathbf{3} & \mathbf{1} \end{matrix}$$

$$\mathbf{H} = \begin{matrix} \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{0} & \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{1} & \mathbf{1} & \mathbf{0} & \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ \mathbf{1} & \mathbf{0} & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} & \mathbf{1} & \mathbf{1} & \mathbf{1} \end{matrix}$$

- The table below shows the unique 6-bit syndrome (in octal) for all possible one and two bit errors. (**Note:** $x^3 + x^6 = 34 + 56 = 011\ 100 \oplus 101\ 110 = 110\ 010 = 62$)

Table 2		$E(x)=0$	$E(x)=x^0$	$E(x)=x^1$	$E(x)=x^2$	$E(x)=x^3$	$E(x)=x^4$	$E(x)=x^5$	$E(x)=x^6$
Error		0000000	0000001	0000010	0000100	0001000	0010000	0100000	1000000
			G	F	E	D	C	B	A
0000000		00	11	23	45	34	67	72	56
0000001	G	11	--	32	54	25	76	63	47
0000010	F	23	32	--	66	17	44	51	75
0000100	E	45	54	66	--	71	22	37	13
0001000	D	34	25	17	71	--	53	46	62
0010000	C	67	76	44	22	53	--	15	31
0100000	B	72	63	51	37	46	15	--	24
1000000	A	56	47	75	13	62	31	24	--

A Check Matrix that Does *not* Work

- The (6 x 7) check matrix **H** for double-bit error correction is defined from the above table (with $i = 1$ and $i = 2$) as shown below. However, *this choice of rows fails*.

	A	B	C	D	E	F	G
H =	5	7	6	3	4	2	1
	7	3	2	5	6	4	1

H =	1	1	1	0	1	0	0
	0	1	1	1	0	1	0
	1	1	0	1	0	0	1
	1	0	0	1	1	1	0
	1	1	1	0	1	0	0
	1	1	0	1	0	0	1

Note: Here, $i = 2$ row is the square of $i = 1$ row (redundant row).

Example: In a^{6i} column (A), $7 = (5)^2 \rightarrow 101 * 101 = 10001 / 1011 = 111 = 7$. Here, $1011 \rightarrow g(X) = X^3 + X + 1$

Syndrome table:

Table 3		$E(x)=0$	$E(x)=x^0$	$E(x)=x^1$	$E(x)=x^2$	$E(x)=x^3$	$E(x)=x^4$	$E(x)=x^5$	$E(x)=x^6$
Error		0000000	0000001	0000010	0000100	0001000	0010000	0100000	1000000
			G	F	E	D	C	B	A
0000000		00	11	24	46	35	62	73	57
0000001	G	11	--	35	57	24	73	62	46
0000010	F	24	35	--	62	11	46	57	73
0000100	E	46	57	62	--	73	24	35	11
0001000	D	35	24	11	73	--	57	46	62
0010000	C	62	73	46	24	57	--	11	35
0100000	B	73	62	57	35	46	11	--	24
1000000	A	57	46	73	11	62	35	24	--

Note: In the above table many different error pattern results in same syndrome.

Example: Error pattern (0000001), (00001010), (0110000), (1000100) results in same syndrome 11.

- The algorithm for building an error control code with a desired Hamming distance (D) using this matrix approach shall be:
 - Start with ($i = 1$);
 - Choose as many rows as necessary ($i = 2, 3, 4, \dots, D - 1$) to achieve the distance (D).
 - Discard the rows that are redundant, and;
 - Build the check matrix.

Examples with $P(x) = x^3 + x + 1$

- For ($D = 3$) start with ($i = 1$), choose additional row ($i = 2$) then keep only row ($i = 1$).
- For ($D = 5$) start with ($i = 1$), choose additional rows ($i = 2, 3, 4$) then keep only rows ($i = 1, 3$) as the others are redundant.
- For ($D = 7$) start with ($i = 1$), choose additional rows ($i = 2, 3, 4, 5, 6$) then keep only rows ($i = 1, 3$) as the others are redundant.
- It is interesting to note that $D = 7$ was already achieved in the first example on this page where the two rows ($i = 1, 3$) gave a (7,1) code describing the trivial case of two codewords (0000000) and (1111111) which actually have distance ($D = 7$). Use of a higher degree polynomial $P(x)$ will lead to more realistic results.

Reed Solomon (RS) Codes:

- Reed-Solomon (RS) codes are a subset of BCH codes, based on extended Galois fields $GF(2^m)$. The symbol-based RS codes boast inherent burst-error correction capability.
- Here, n , k , and r are multi-bit symbols. i.e, for $d_{\min} = 2 \rightarrow$ error detection of 1 multi-bit symbol.
- For $GF(2^3)$, n , k , and r represents 3-bit symbols.
- The general form of the RS code generator polynomial $G(x)$ is:
 - $G(x) = (x - a^1) (x - a^2) (x - a^3) (x - a^4) \dots (x - a^{2t})$

- A $GF(2^3)$ polynomial can be defined using the above 3-bit values in $\{a^n\}$ as coefficients.
- For example, this polynomial $M(x)$ has 3-bit coefficients in the range 0 to 7:
 - $M(x) = a^5 x^3 + a^3 x^2 + a^6 x + a^6$
 $= 7 x^3 + 3 x^2 + 5 x + 5$, *also written as (7 3 5 5)*
 $= (111) x^3 + (011) x^2 + (101) x + (101)$
 $= \text{the 12-bit binary value } 111011101101$

For further detail: [Click here](#)

For primitive polynomial $P(X) = X^3 + X + 1$,

0	a^0	a^1	a^2	a^3	a^4	a^5	a^6
0	1	2	4	3	6	7	5

Example: $a^6 \rightarrow X^6/P(X) = X^2 + 1 = [1 \ 0 \ 1] = 5$

In RS, the coefficient of $G(X)$ and $m(X)$ are 3-bit symbol in $GF(2^3)$.

Example: $m(X) = a^4X^3 + a^0X^2 + a^5X + a^4 = 6X^3 + X^2 + 7X + 6 \rightarrow m = \{6 \ 1 \ 7 \ 6\}$ (from the above table)

RS code is denoted by $(n, n - 2t)$ where t is the number of symbol can be corrected.

Example: For $n = 7$ if $t = 1 \rightarrow (7, 5)$, $t = 2 \rightarrow (7, 3)$

The $G(X)$ in RS is of the form $(X - a^1)(X - a^2)(X - a^3) \dots (X - a^{2t})$

Example: For $t = 1$, $G(X) = (X - a^1)(X - a^2) = (X + 2)(X + 4) = X^2 + 6X + (2 \times 4 = a^1 \times a^2 = a^3 = 3) = X^2 + 6X + 3$ (Note: Refer to the above table)

Similarly, if $t = 2$,

$$\begin{aligned} G(X) &= (X - a^1)(X - a^2)(X - a^3)(X - a^4) = (X + 2)(X + 4)(X + 3)(X + 6) \\ &= X^4 + 3X^3 + X^2 + 2X + 3 \end{aligned}$$

Encoding: $(7, 5)$ with $t = 1$, $G(X) = X^2 + 6X + 3$. Assume $m = [5 \ 1 \ 3 \ 5 \ 4] = 5X^4 + X^3 + 3X^2 + 5X + 4$

To generate systematic codeword:

Step-1: Multiply $m(X)$ by X^2 resulting in $5X^6 + X^5 + 3X^4 + 5X^3 + 4X^2$

Step-2: Divide Step-1 by $G(X)$. Remainder = $[6 \ 4]$ ([Calculation](#))

Step-3: Join Step-1 and Step-2 resulting in $c(X) = 5X^6 + X^5 + 3X^4 + 5X^3 + 4X^2 + 6X + 4 =$
 $[5 \ 1 \ 3 \ 5 \ 4 \ 6 \ 4]$

Decoding: $G(X) = X^2 + 6X + 3$. Assume $m = [5\ 1\ 3\ 5\ 4] = 5X^4 + X^3 + 3X^2 + 5X + 4$ and

$c(X) = 5X^6 + X^5 + 3X^4 + 5X^3 + 4X^2 + 6X + 4 = [5\ 1\ 3\ 5\ 4\ 6\ 4]$

$s(X) = c'(X) / G(X) = [0\ 0] \rightarrow$ No error

If $c'(X) = [5\ 1\ 2\ 5\ 4\ 6\ 4]$

$s(X) = [7\ 3] \rightarrow$ Error in X^4

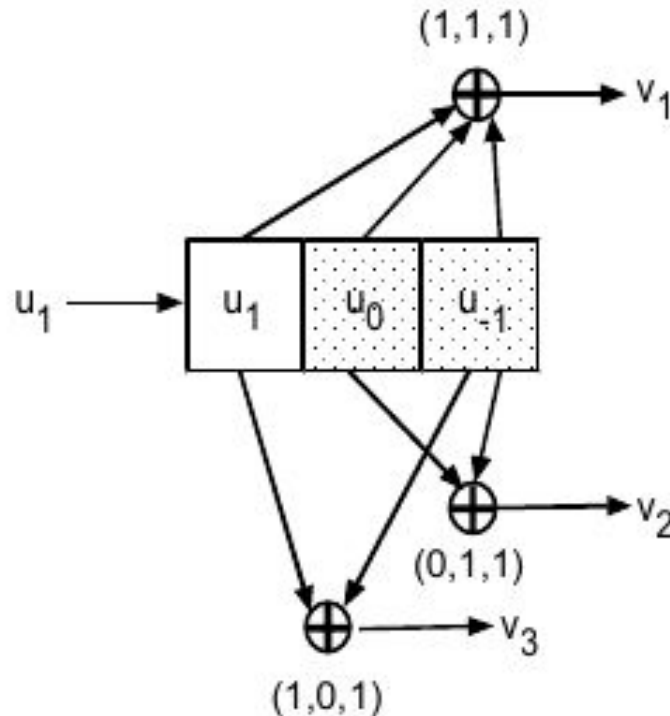
The error position X^4 in this table w.r.t. row 1 indicate, the correct coefficient is $2 + 1 = 010 + 001 = 011 = 3$.

	x^6	x^5	x^4	x^3	x^2	x^1	x^0
1	(6 2)	(7 2)	(7 3)	(1 1)	(6 3)	(1 0)	(0 1)
2	(7 4)	(5 4)	(5 6)	(2 2)	(7 6)	(2 0)	(0 2)
3	(1 6)	(2 6)	(2 5)	(3 3)	(1 5)	(3 0)	(0 3)
4	(5 3)	(1 3)	(1 7)	(4 4)	(5 7)	(4 0)	(0 4)
5	(3 1)	(6 1)	(6 4)	(5 5)	(3 4)	(5 0)	(0 5)
6	(2 7)	(4 7)	(4 1)	(6 6)	(2 1)	(6 0)	(0 6)
7	(4 5)	(3 5)	(3 2)	(7 7)	(4 2)	(7 0)	(0 7)

Convolution coding

- Convolutional codes are commonly specified by three parameters; (n, k, m) .
 - n = number of output bits
 - k = number of input bits
 - m = number of memory registers
- The quantity k/n called the **code rate**, is a measure of the efficiency of the code. Commonly k and n parameters range from 1 to 8, m from 2 to 10 and the code rate from $1/8$ to $7/8$ except for deep space applications where code rates as low as $1/100$ or even longer have been employed.
- Often the manufacturers of convolutional code chips specify the code by parameters (n, k, L) , The quantity L is called the **constraint length** of the code and is defined by $L = k(m-1)$

- The constraint length L represents the number of bits in the encoder memory that affect the generation of the n output bits.
- **Example:** (3, 1, 3) convolution code



Generator Polynomial:

$$v_1 = \text{mod-2 } (u_1 + u_0 + u_{-1})$$

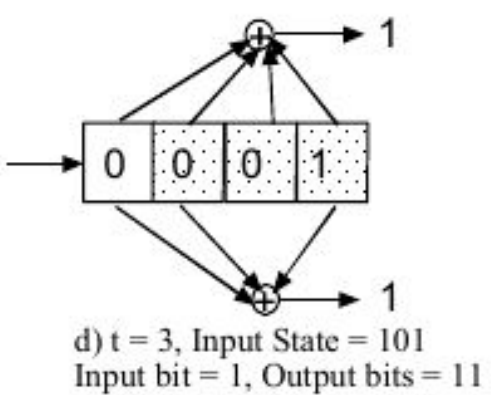
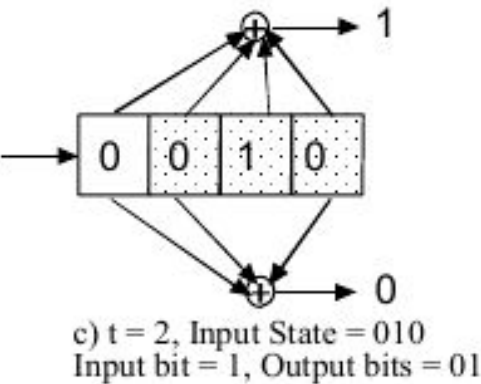
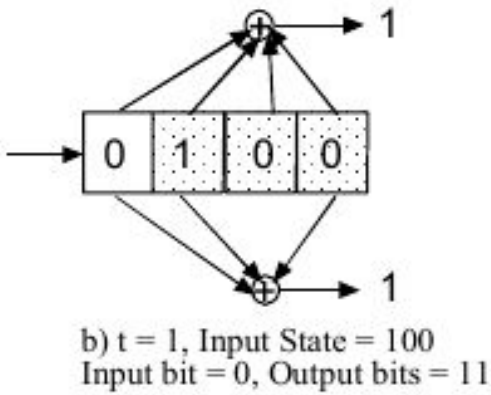
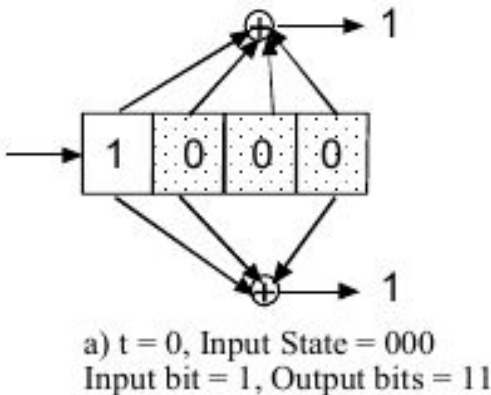
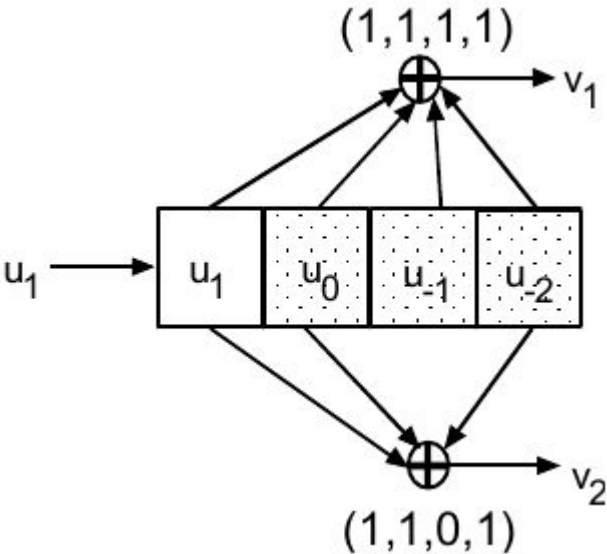
$$v_2 = \text{mod-2 } (u_0 + u_{-1})$$

$$v_3 = \text{mod-2 } (u_1 + u_{-1})$$

Generator polynomials for code rate $\frac{1}{2}$

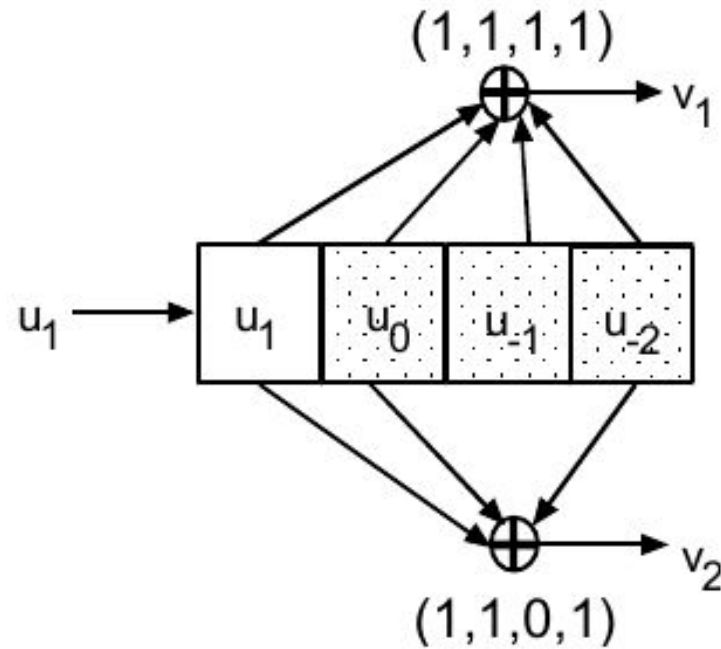
Constraint Length	G₁	G₂
3	110	111
4	1101	1110
5	11010	11101
6	110101	111011
7	110101	110101
8	110111	1110011
9	110111	111001101
10	110111001	1110011001

Encoding two bit sequence of 10 with the (2, 1, 4) code

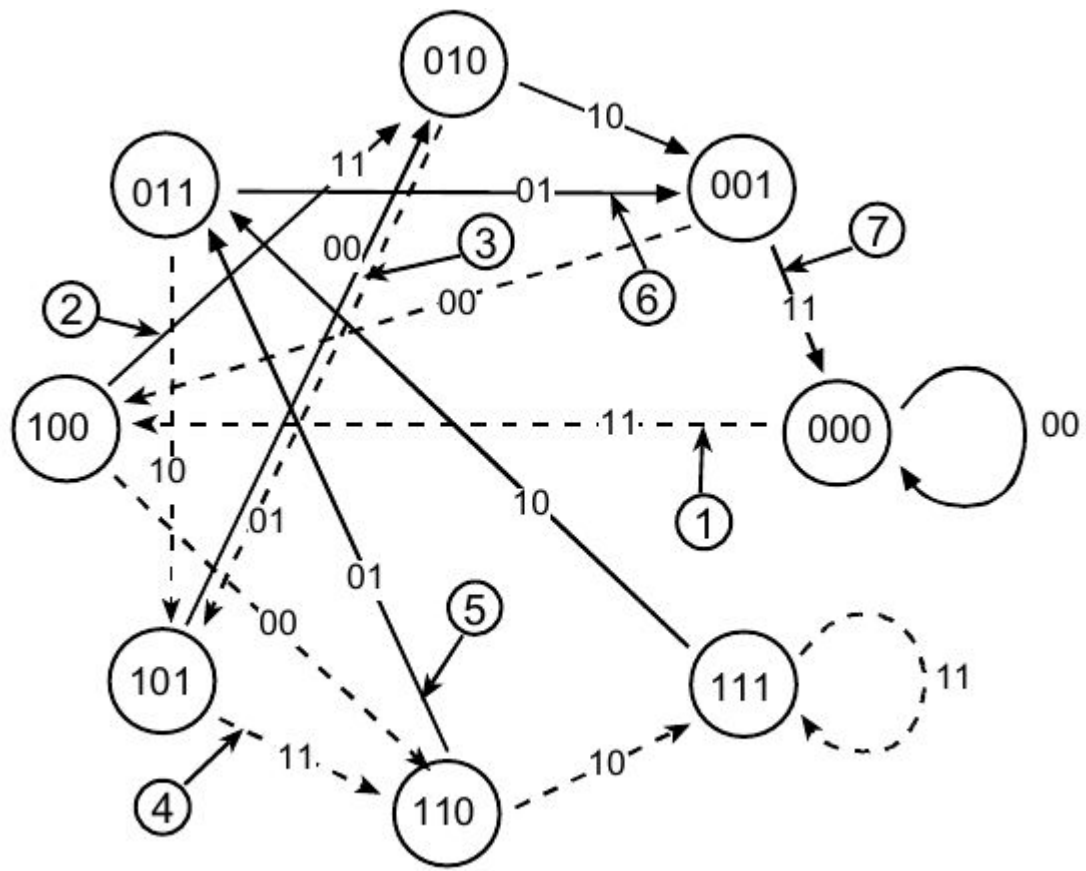


Convolution encoding can be presented in,

- State Diagram
- Tree Diagram
- Trellis Diagram



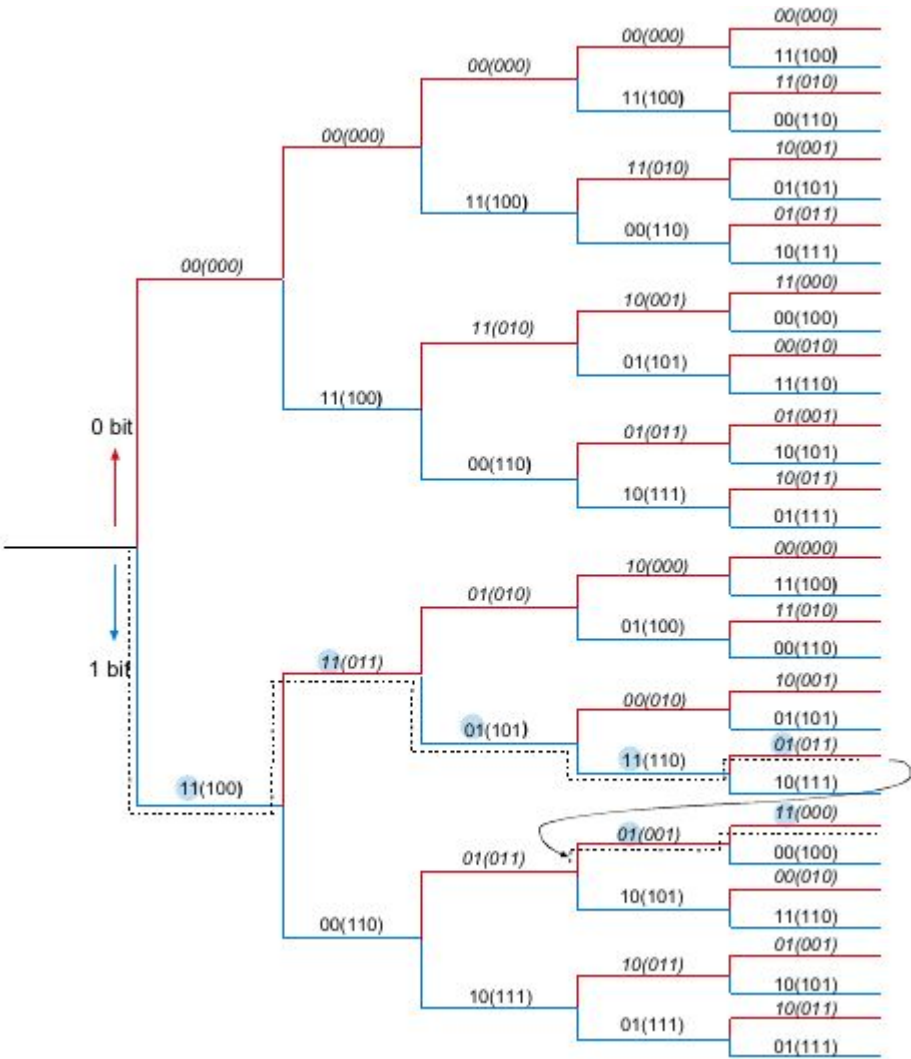
State diagram of (2, 1, 4) code



Encoding of 1011:

11 11 01 11 01 01 11

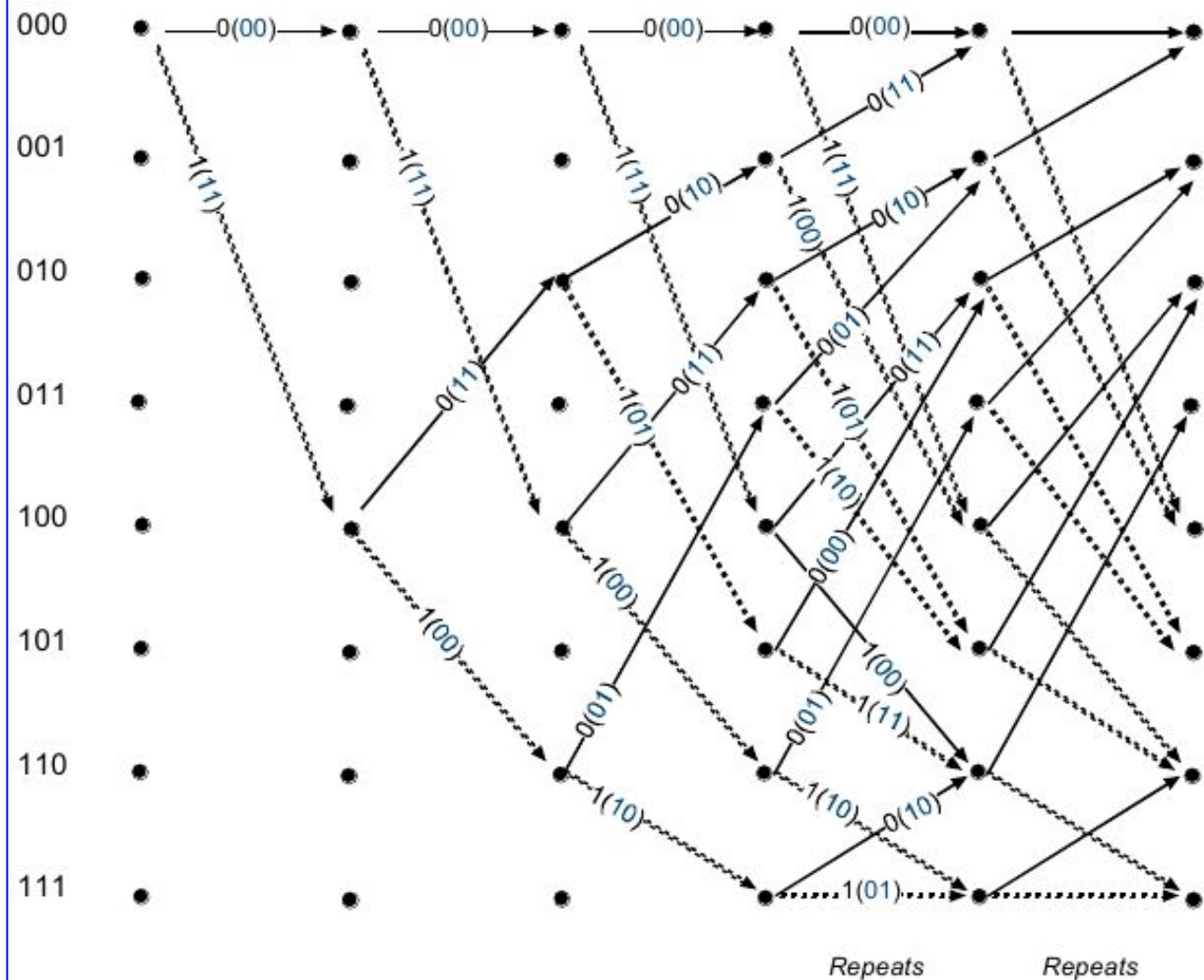
Tree diagram of (2, 1, 4) code



Trellis diagram of (2, 1, 4)

Code:

- The x-axis is discrete time and all possible states are shown on the y-axis.
- We move horizontally through the trellis with the passage of time.
- Each transition means new bits have arrived.



Convolutional Decoding

There are several different approaches to decoding of convolutional codes. These are grouped in two basic categories.

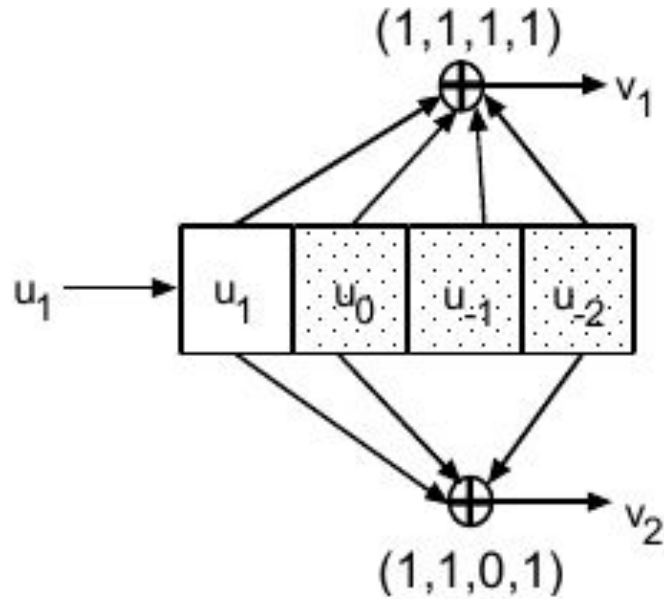
- Sequential Decoding
 - Fano algorithm
- Maximum likelihood decoding
 - Viterbi decoding

Example: (2, 1, 4) convolution encoder

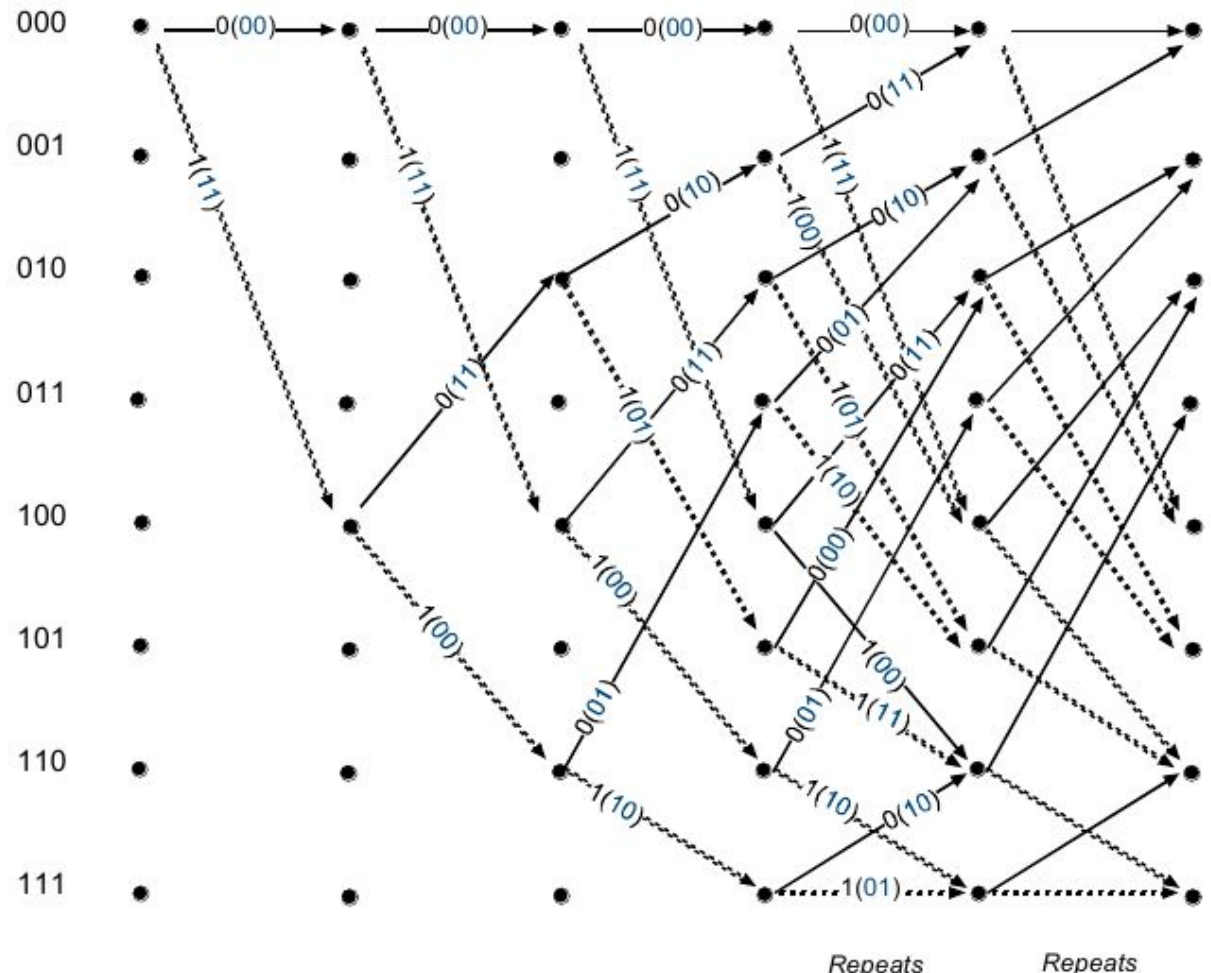
Data: 1011

Codeword: 11 11 01 11 01 01 11

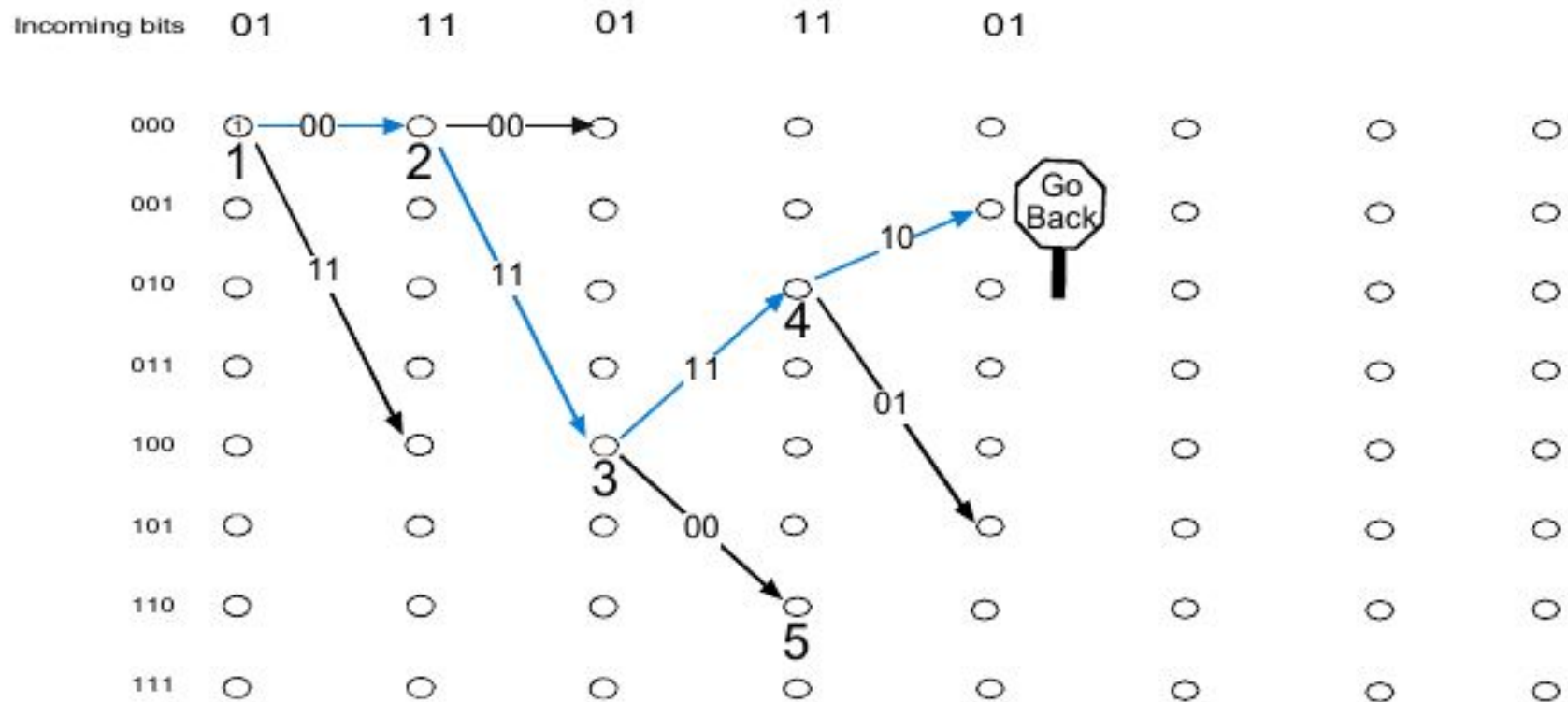
1-bit error: **0**1 11 01 11 01 01 11



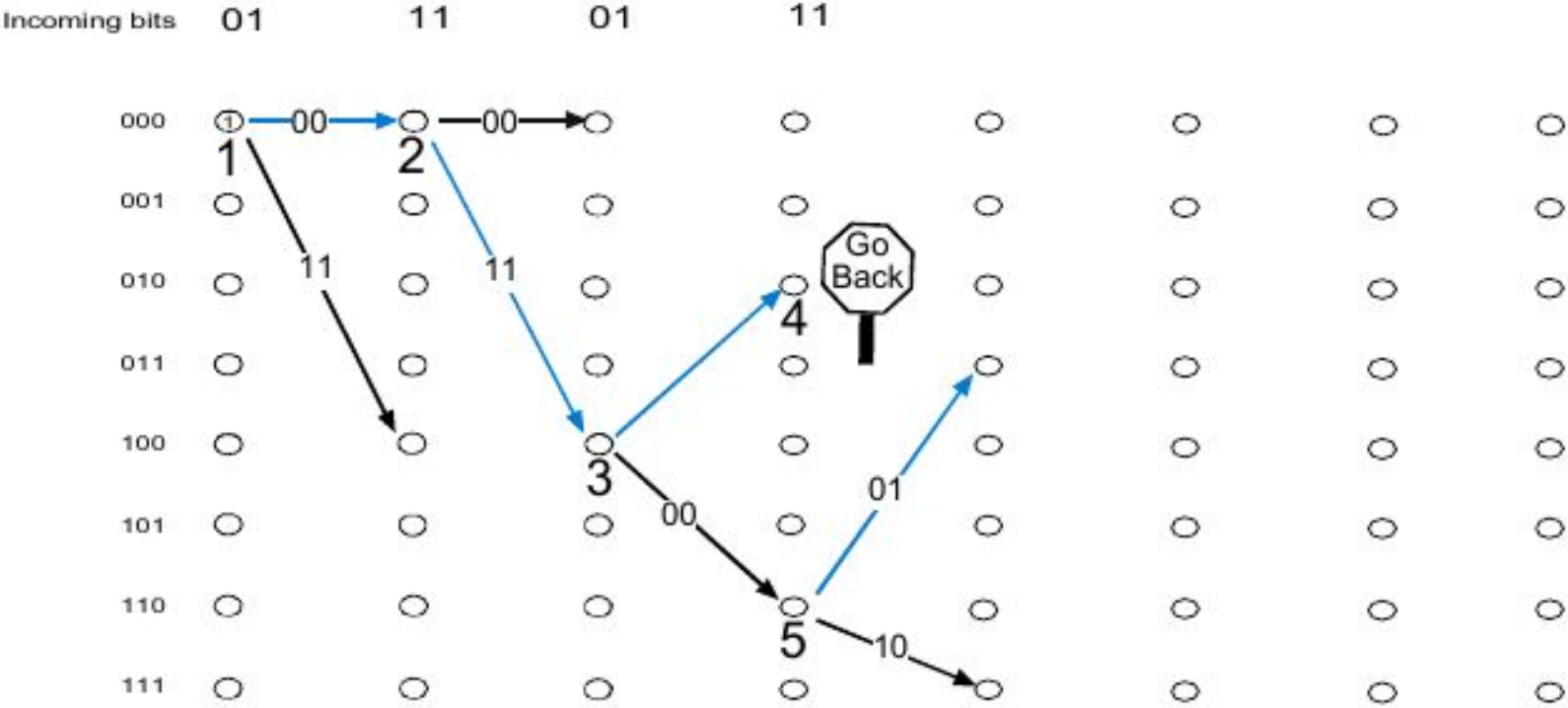
Trellis diagram of (2, 1, 4) encoding



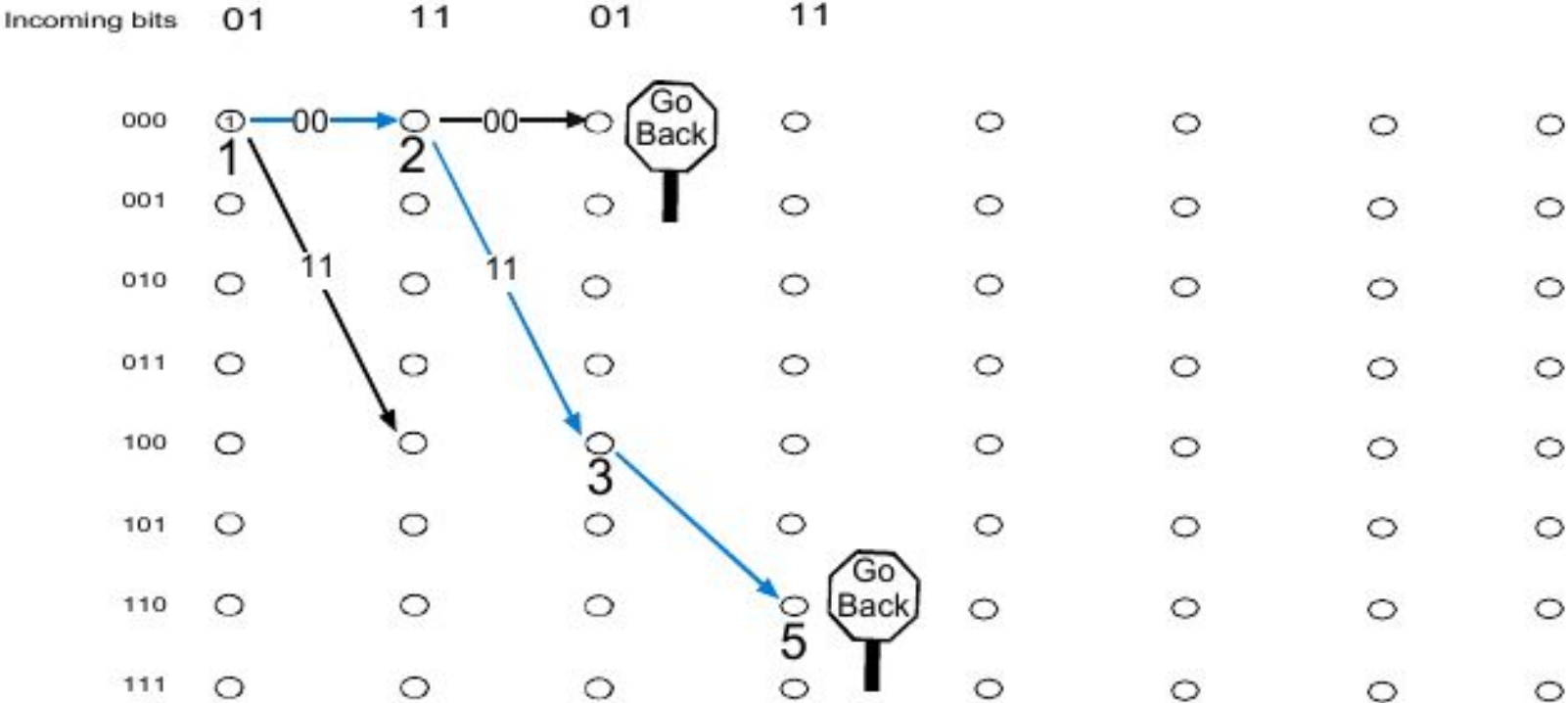
Sequential Decoding



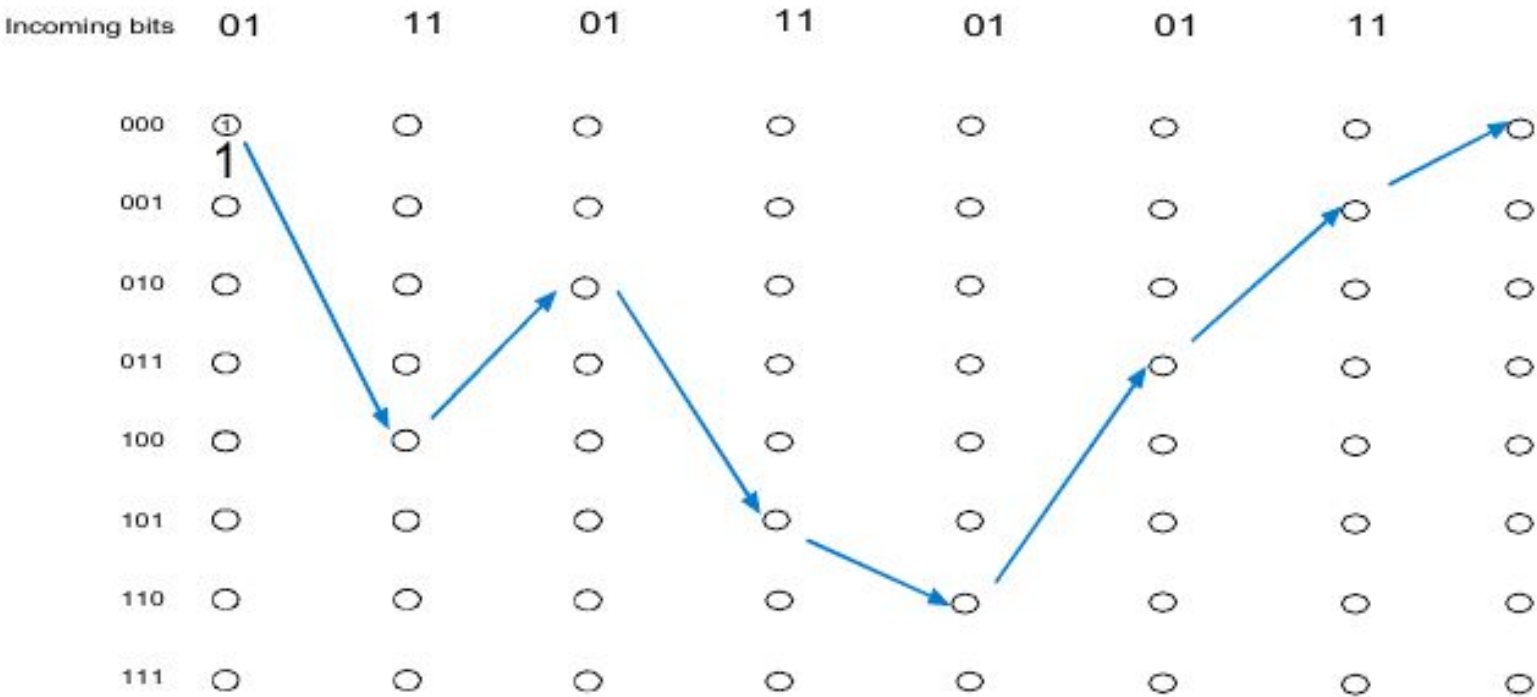
Sequential decoding



Sequential decoding



Sequential decoding



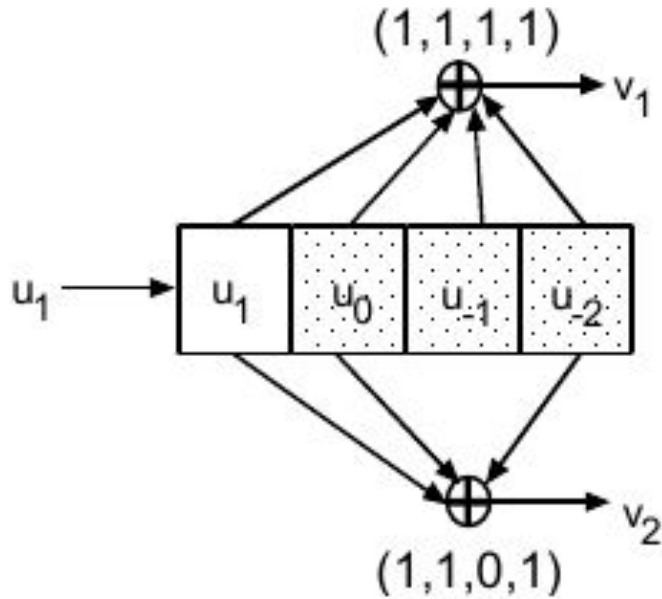
Maximum Likelihood - Viterbi decoding

Example: (2, 1, 4) convolution encoder

Data: 1011

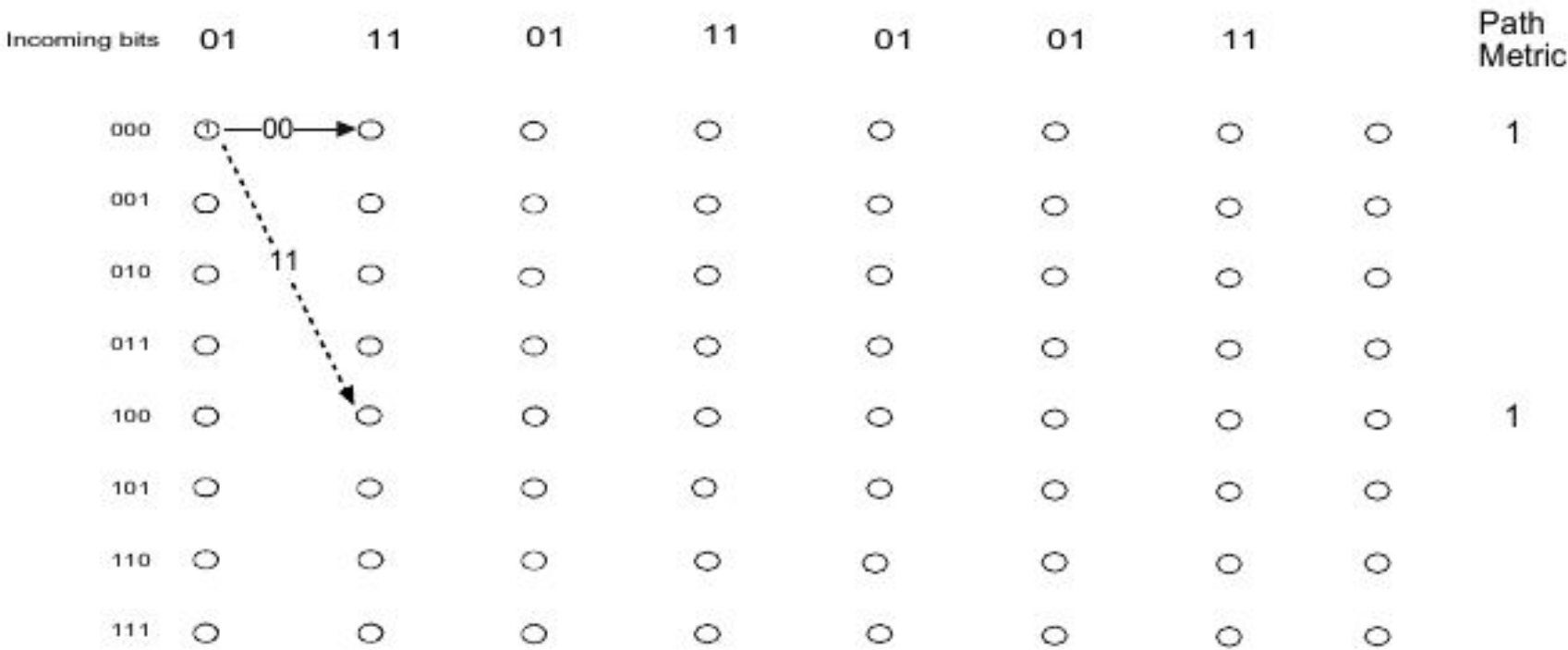
Codeword: 11 11 01 11 01 01 11

1-bit error: **0**1 11 01 11 01 01 11

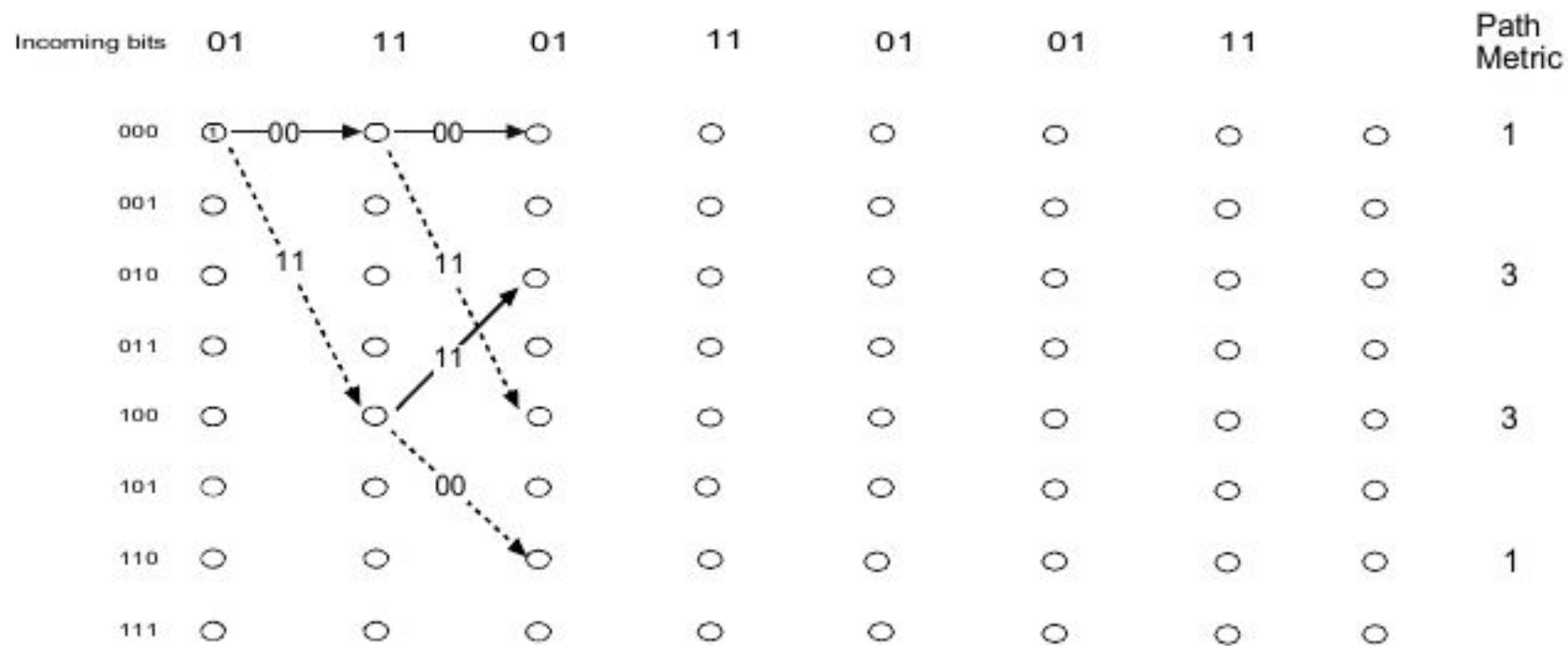


Maximum Likelihood - Viterbi decoding

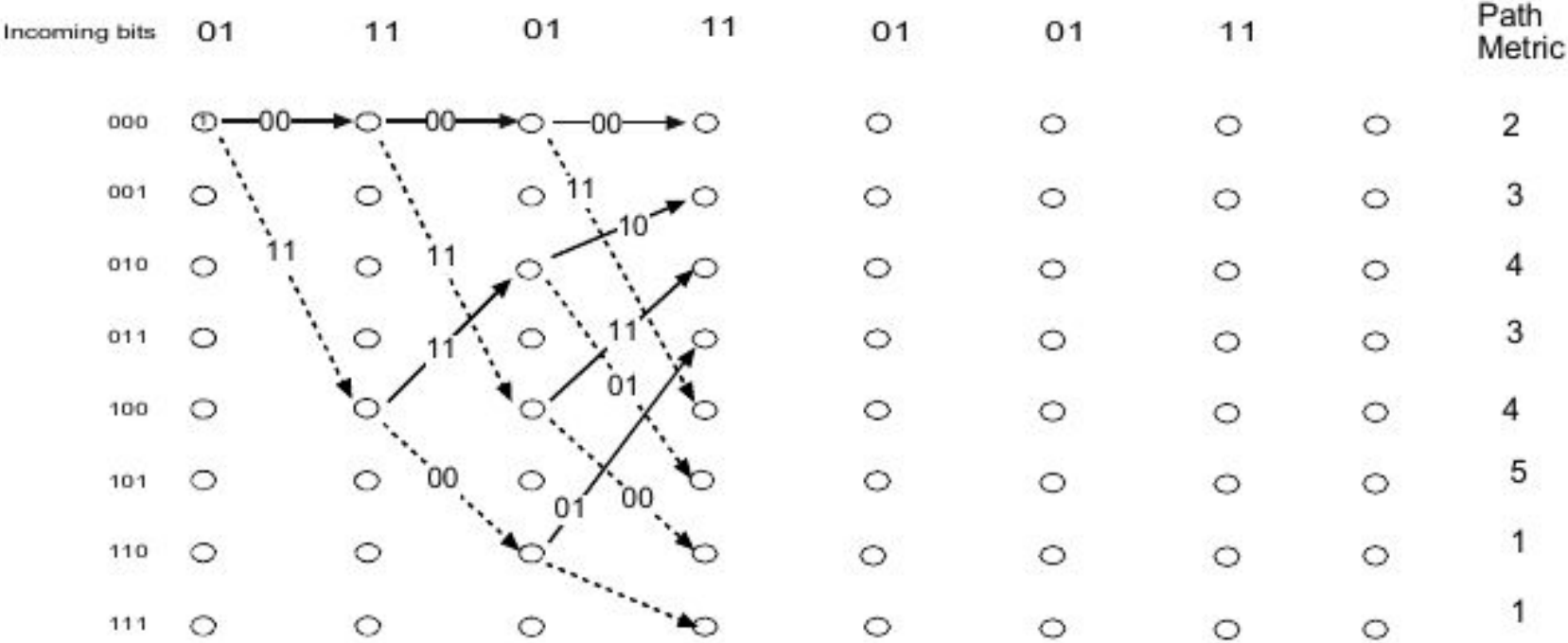
Step-1:



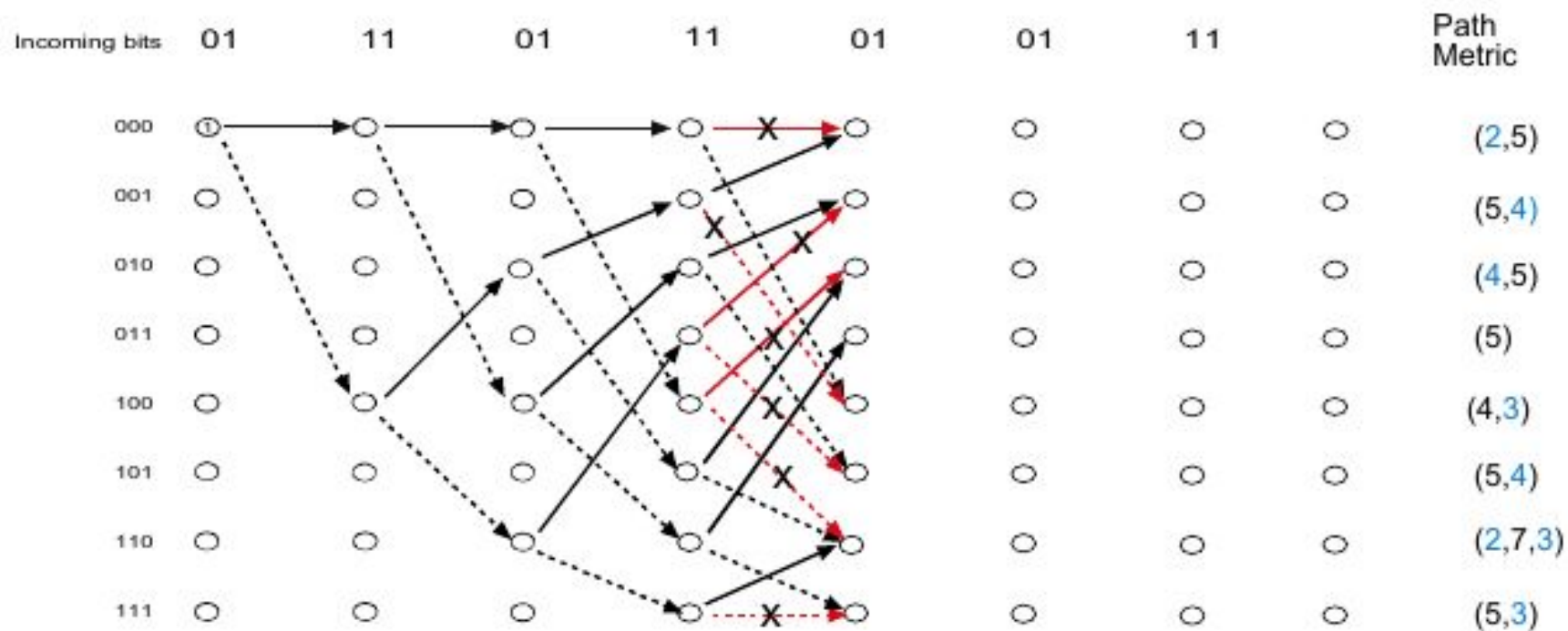
Step-2:



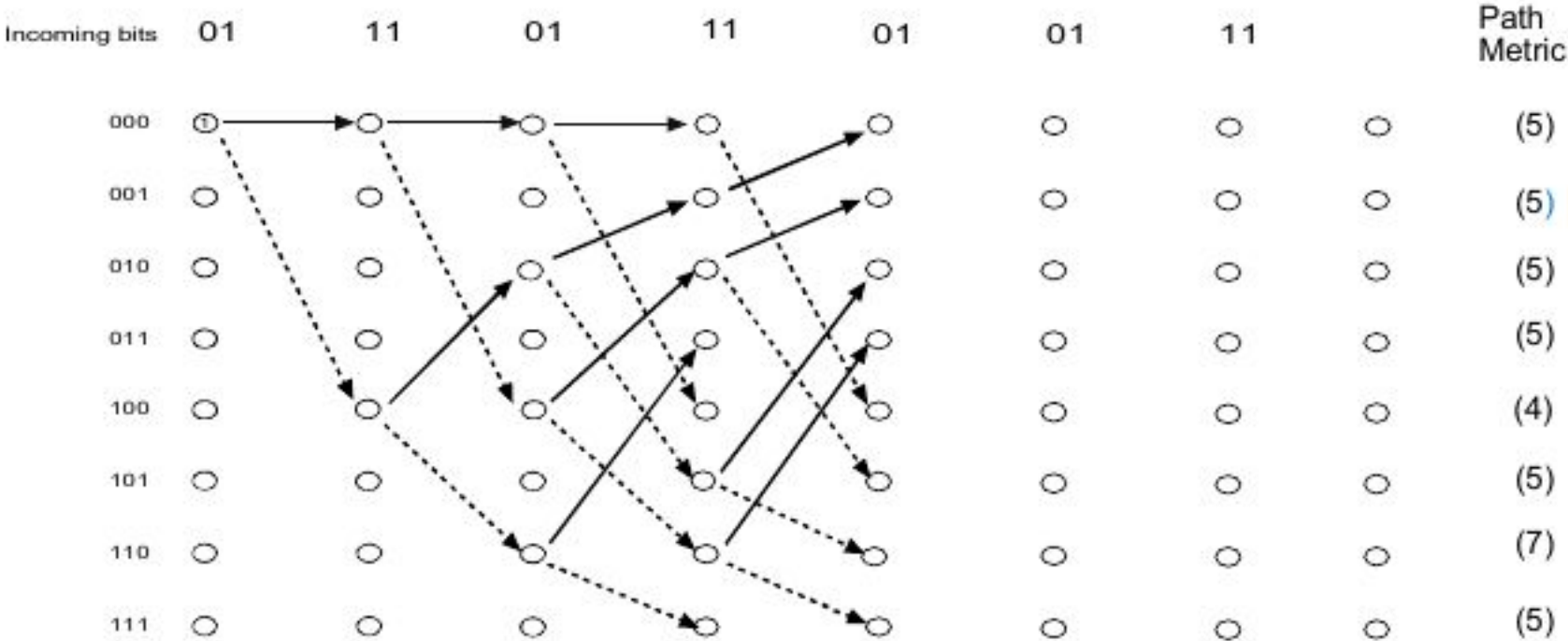
Step-3:



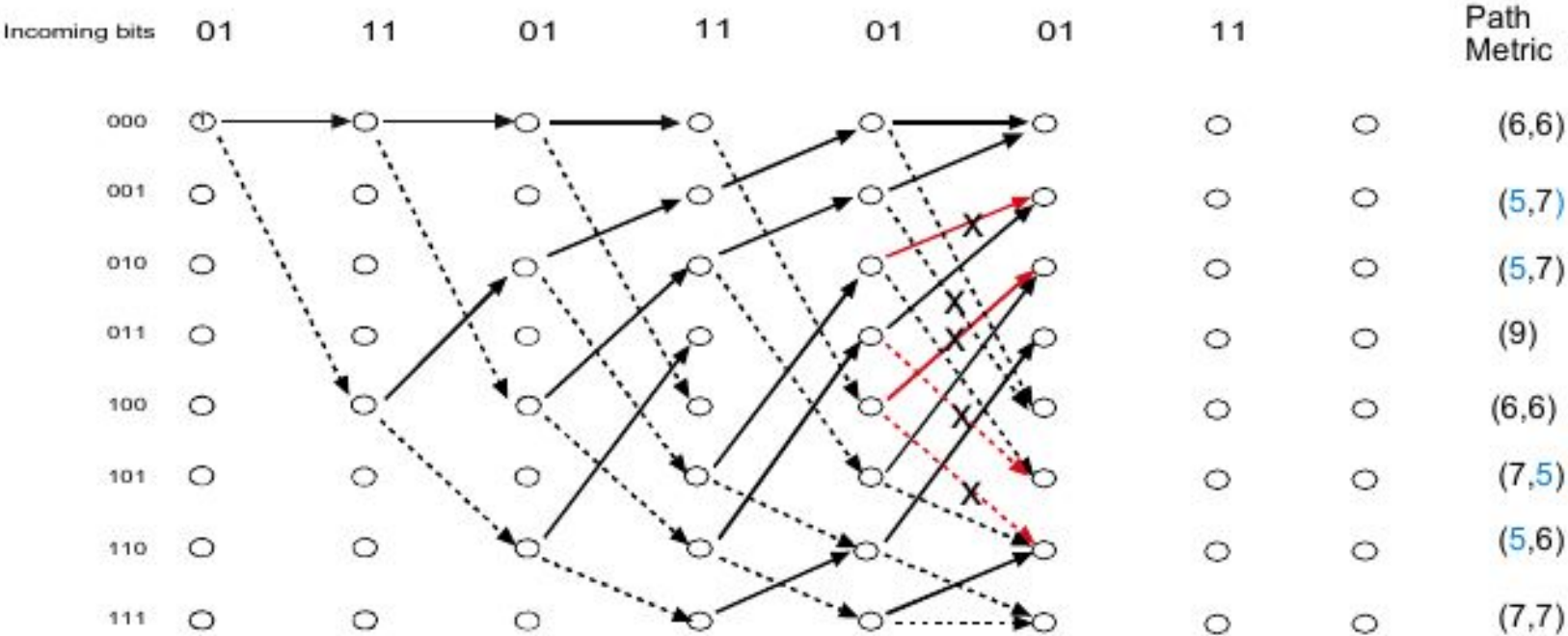
Step-4:



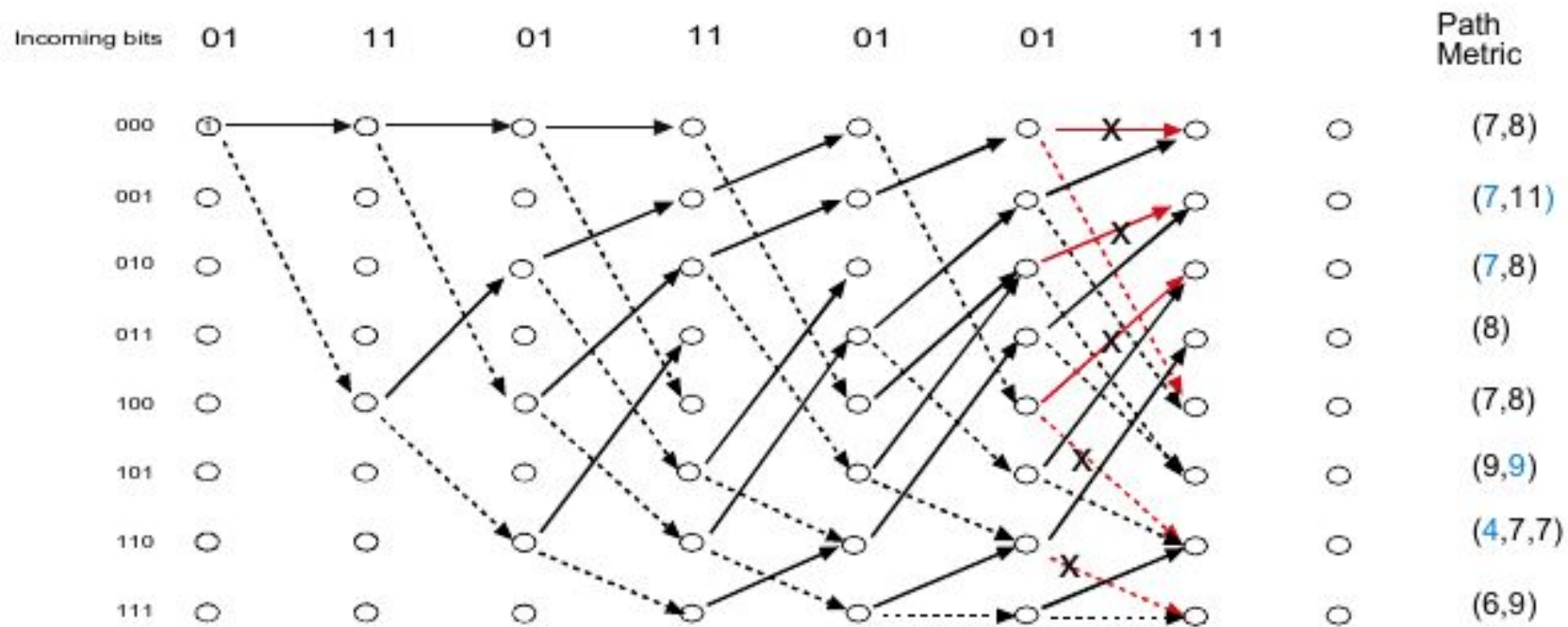
Step-4 (After discarding)



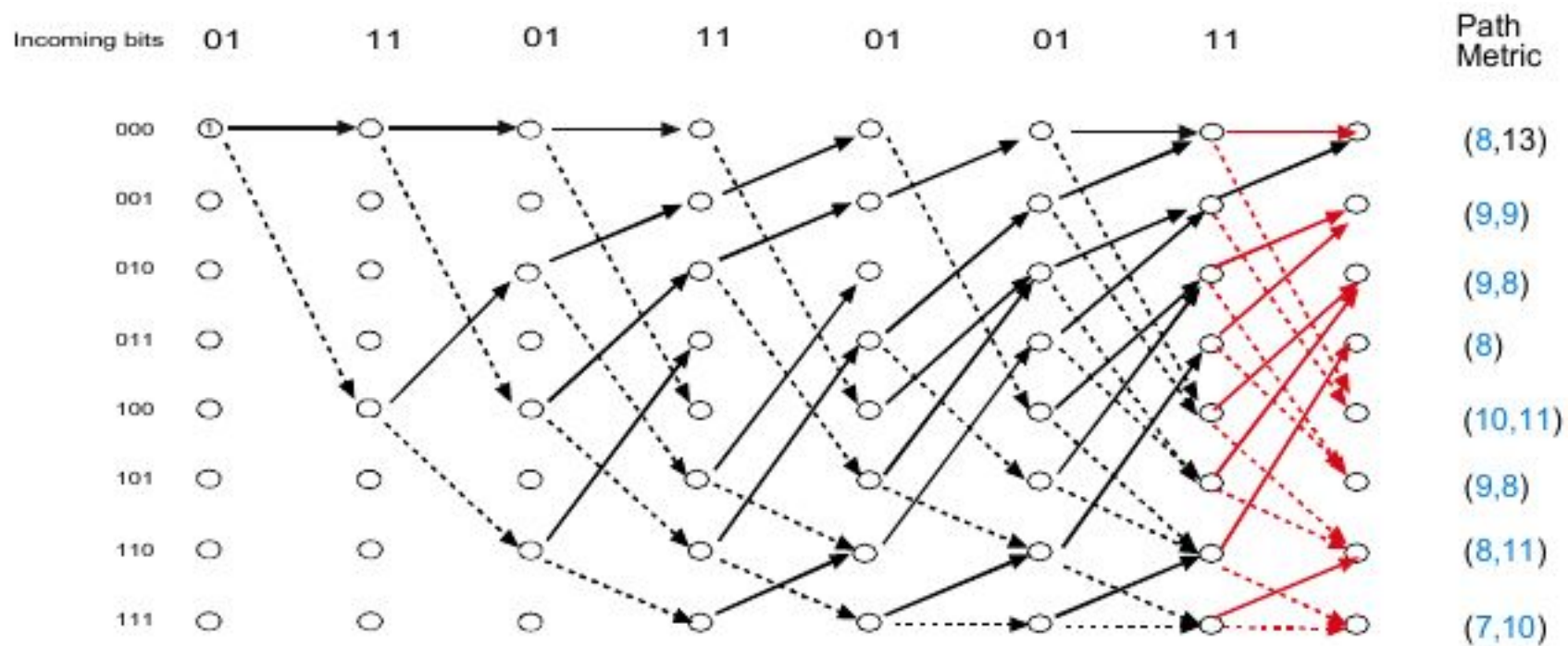
Step-5:



Step-6:

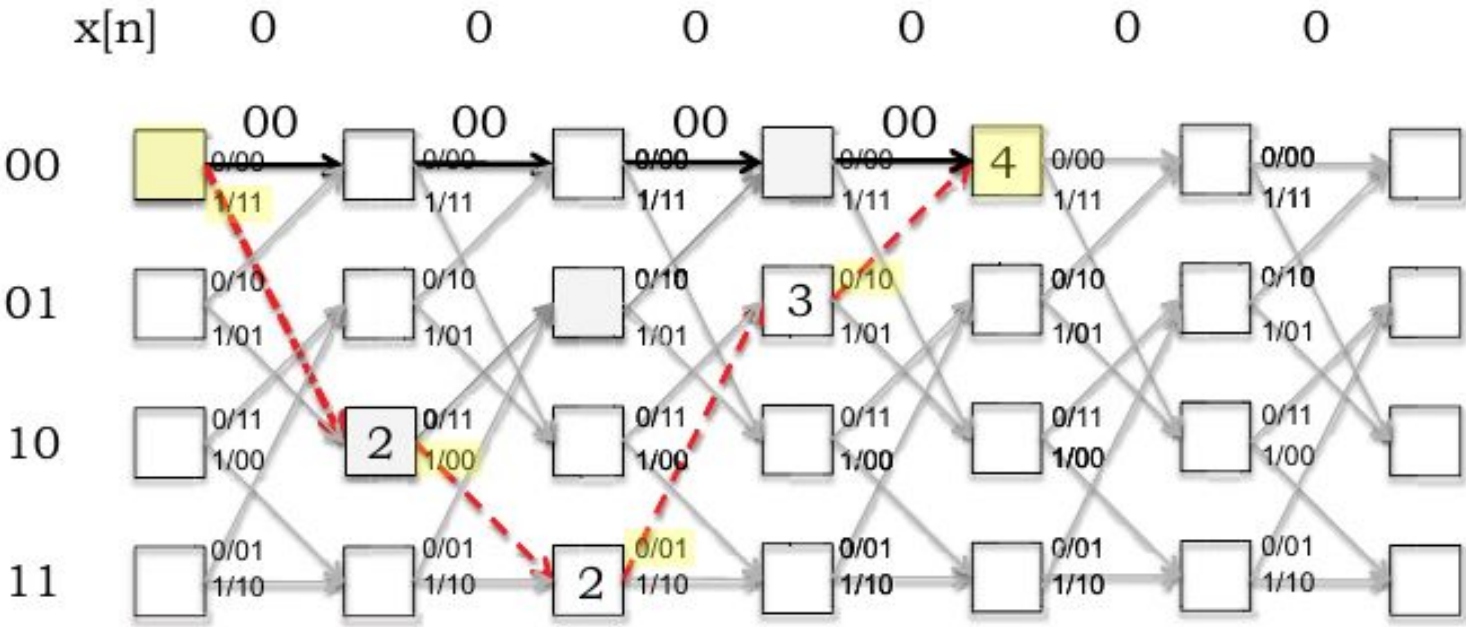


Step-7

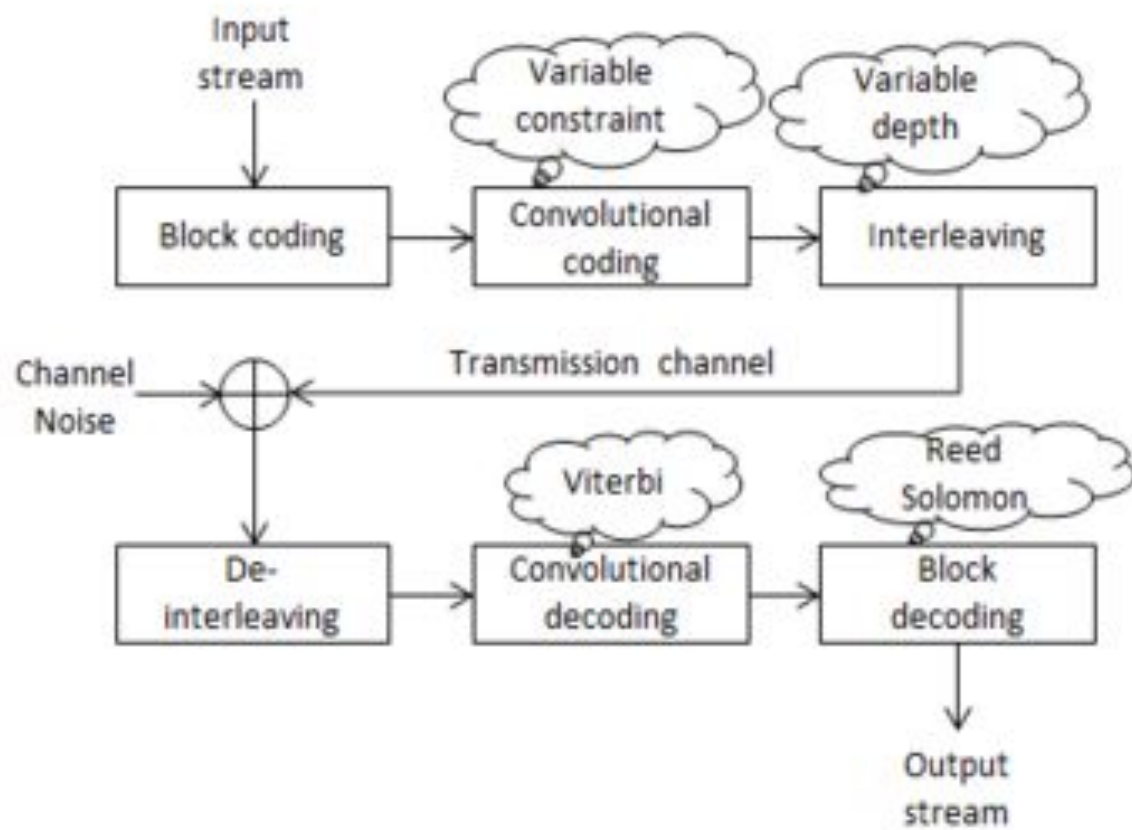


Minimum Hamming distance

- Find path with smallest non-zero path metric going from first 00 state to a future 00 state.



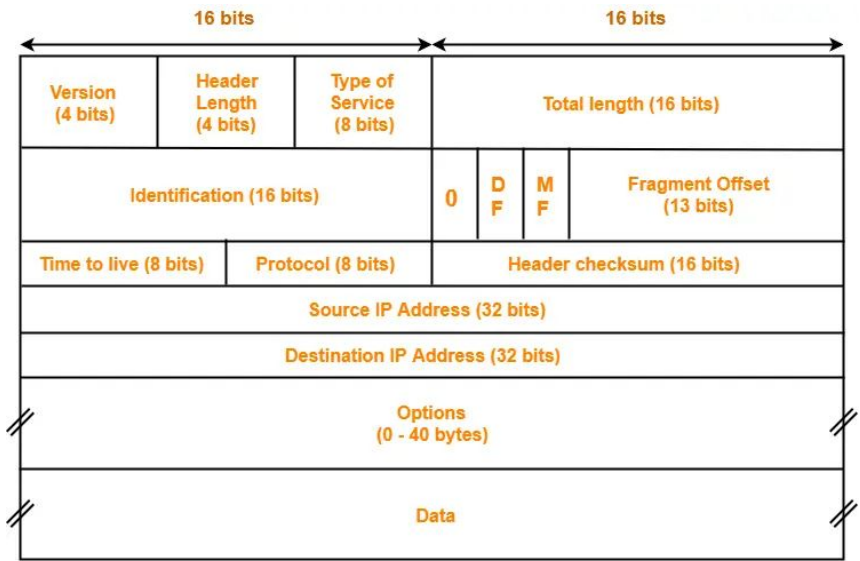
- Here, $d_{min} = 4$, so can correct 1 error in 8 bits:



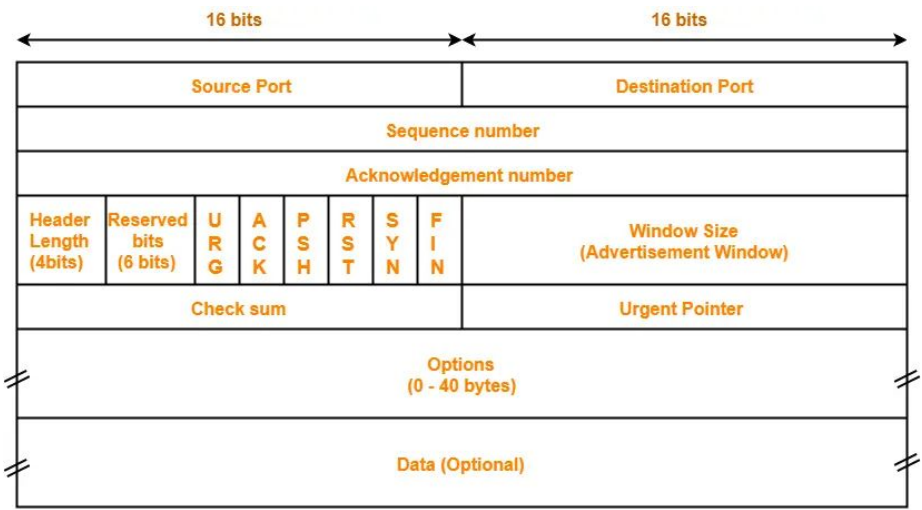
A Typical FEC, Forward Error Correction Sequence

Checksum

- Checksum is an error-detecting technique that can be applied to a message of any length.
- In the Internet, the checksum technique is mostly used at the network and transport layer rather than the data-link layer.

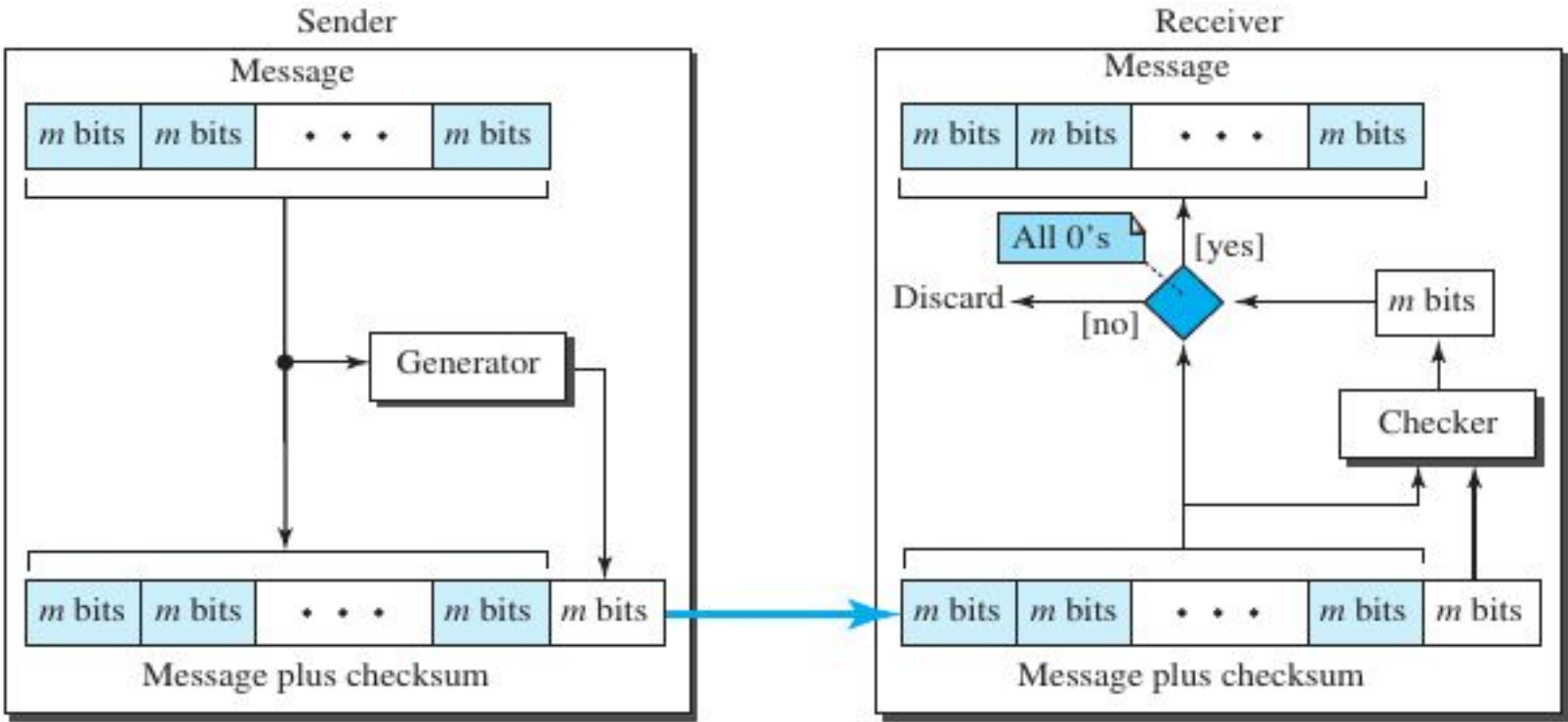


IPv4 Header



TCP Header

Checksum at sender and receiver

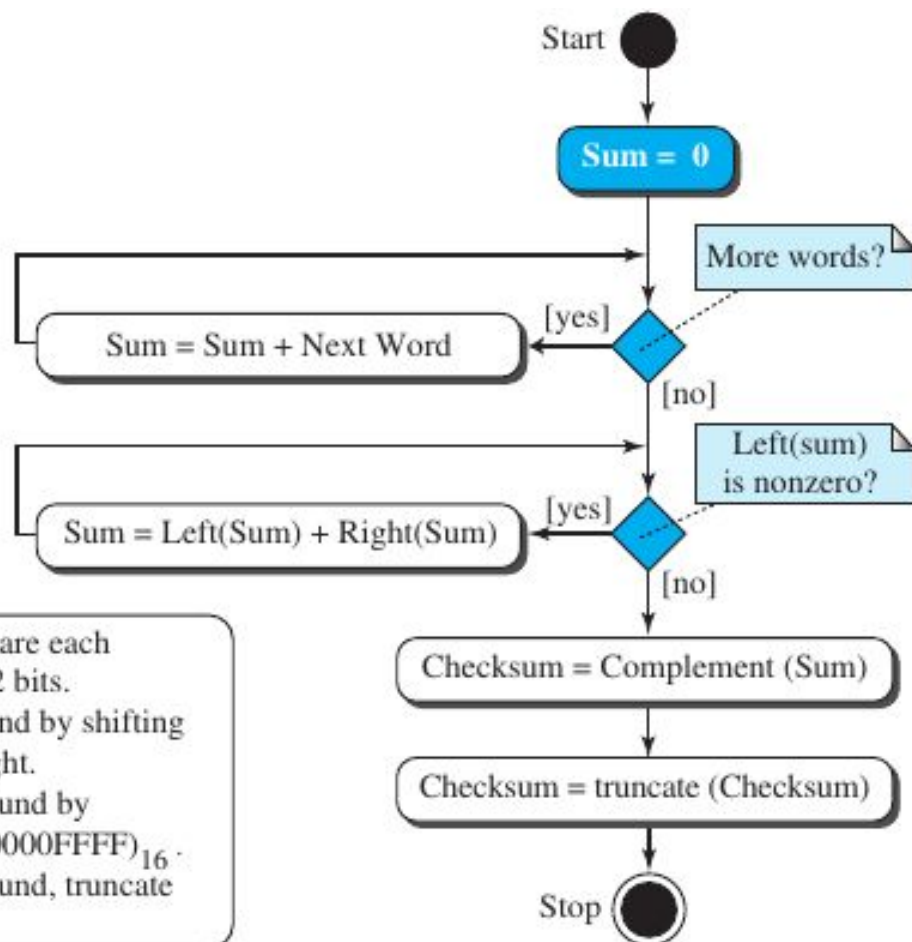


Checksum calculation for data {7, 11, 12, 0, 6}:

- **Case-1:** Sum (36) of all is the checksum at the sender. So, the transmitted data is {7, 11, 12, 0, 6, 36}. At the receiver find the sum of first five numbers and compare it with checksum.
- **Case-2:** Using 1's complement
 - Every data is represented in 'm' bit form including checksum. If the calculated checksum is more than 'm' bit, the leftmost bits are added to the rightmost 'm' bits.
 - In the above example, first five numbers can be represented by 4-bit, so the checksum 36 (100100_2) is represented by $(10)_2 + (0100)_2 = (0110)_2 = 6$

- **Case-3:** Complement the checksum
 - To simplify the receiver work, send the complement of the checksum calculated in Case-2.
 - i.e, send the checksum as 9 (1001_2) which is the complement of 6 (0110_2).
 - At the receiver add $\{7, 11, 12, 0, 6, 9\} \rightarrow 45 \rightarrow (1011010)_2$. After wrapping to 4-bit $(1011010)_2 \rightarrow (1111)_2$. In 1's complement is 0.

16-bit checksum calculation:



Notes:

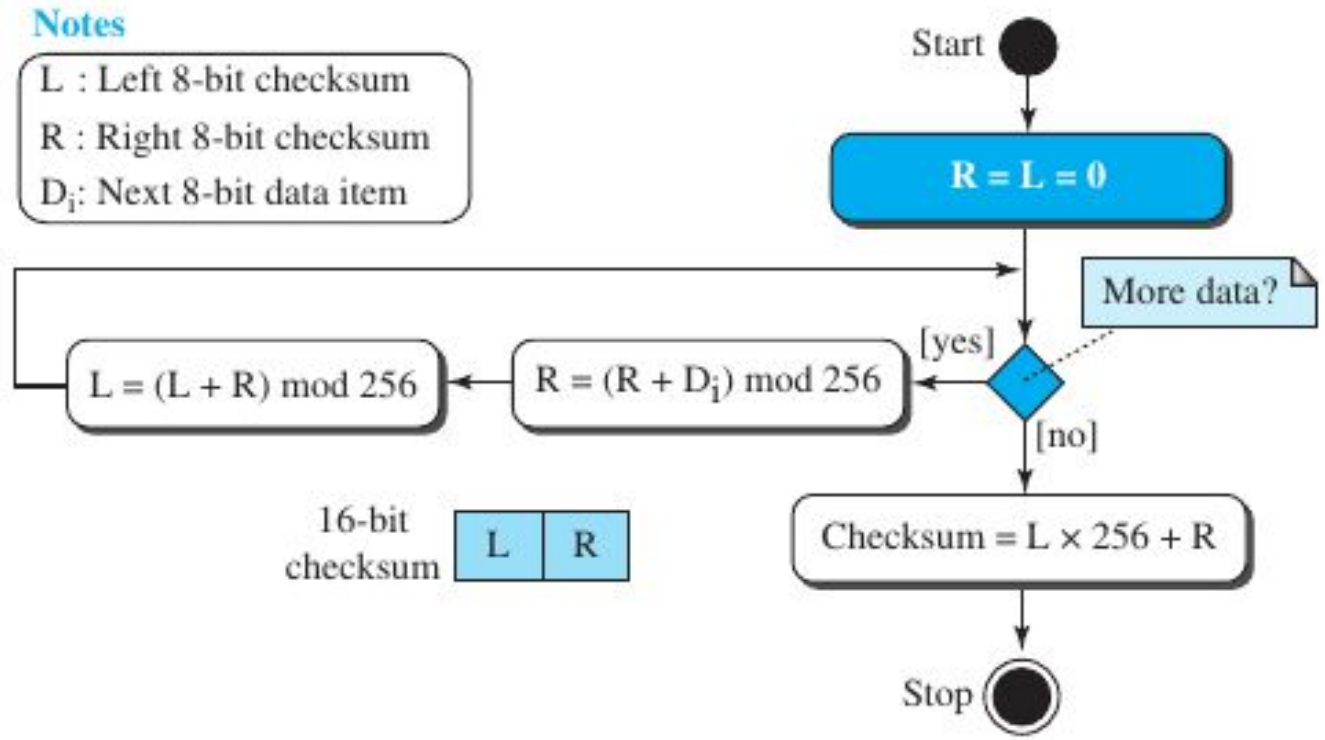
- Word and Checksum are each 16 bits, but Sum is 32 bits.
- Left(Sum) can be found by shifting Sum 16 bits to the right.
- Right(Sum) can be found by ANDing Sum with $(0000FFFF)_{16}$.
- After Checksum is found, truncate it to 16 bits.

Drawback with traditional checksum

- However, it is not as strong as the CRC in error-checking capability.
- For example, if the value of one word is incremented and the value of another word is decremented by the same amount, the two errors cannot be detected because the sum and checksum remain the same.
- Also, if the values of several words are incremented but the sum and the checksum do not change, the errors are not detected.
- If two 16-bit items are transposed in transmission, the checksum cannot catch this error. The reason is that the traditional checksum is not weighted: it treats each data item equally.

Fletcher checksum

- The Fletcher checksum was devised to weight each data item according to its position.



Q1.

A sender needs to send the four data items 0x3456, 0xABCC, 0x02BC, and 0xEEEE.
Answer the following:

- Find the checksum at the sender site.
- Find the checksum at the receiver site if there is no error.
- Find the checksum at the receiver site if the second data item is changed to 0xABCE.
- Find the checksum at the receiver site if the second data item is changed to 0xABCE and the third data item is changed to 0x02BA.

a. Checksum at the sender site

1	2	2	2	Carries
3	4	5	6	
A	B	C	C	
0	2	B	C	
E	E	E	E	
0	0	0	0	Checksum (initial)
D	1	C	C	Sum (partial)
D	1	C	D	Sum
2	E	3	2	Checksum (to send)

Diagram details: A red line starts at the top of column 1, goes down, then right to a '1' in the 'Carries' column between the 'Sum (partial)' and 'Sum' rows. Another red line starts at the top of column 1, goes down, then right to a '1' in the 'Carries' column between the 'Checksum (initial)' and 'Checksum (to send)' rows.

b. Checksum at the receiver site (no error)

1	2	2	2	Carries
3	4	5	6	
A	B	C	C	
0	2	B	C	
E	E	E	E	
2	E	3	2	Checksum (received)
F	F	F	E	Sum (partial)
F	F	F	F	Sum
0	0	0	0	Checksum

Diagram details: A red line starts at the top of column 1, goes down, then right to a '1' in the 'Carries' column between the 'Sum (partial)' and 'Sum' rows. Another red line starts at the top of column 1, goes down, then right to a '1' in the 'Carries' column between the 'Checksum (received)' and 'Checksum' rows.

2	3	3	3	Carries
3	4	5	6	
A	B	C	E	← Error here
0	2	B	C	
E	E	E	E	
2	E	3	2	Checksum (received)
0	0	0	0	Sum (partial)
0	0	0	2	Sum
F	F	F	D	Checksum

c. Checksum at the receiver site (one caught error)

1	2	2	2	Carries
3	4	5	6	
A	B	C	E	— Error here
0	2	B	A	— Error here
E	E	E	E	
2	E	3	2	Checksum (received)
F	F	F	E	Sum (partial)
F	F	F	F	Sum
0	0	0	0	Checksum

d. Checksum at the receiver site (two errors, but not caught)

Q. What is the maximum effect of a 2-ms burst of noise on data transmitted at the following rates?

- a. 1500 bps b. 12 kbps c. 100 kbps d. 100 Mbps

Ans:

- | | | | |
|-----------|-----------------|---|--------------------------|
| a. | vulnerable bits | $= (1,500) \times (2 \times 10^{-3})$ | $= 3 \text{ bits}$ |
| b. | vulnerable bits | $= (12 \times 10^3) \times (2 \times 10^{-3})$ | $= 24 \text{ bits}$ |
| c. | vulnerable bits | $= (100 \times 10^3) \times (2 \times 10^{-3})$ | $= 200 \text{ bits}$ |
| d. | vulnerable bits | $= (100 \times 10^6) \times (2 \times 10^{-3})$ | $= 200,000 \text{ bits}$ |

Q.

We need a dataword of at least 11 bits. Find the values of k and n in the Hamming code $C(n, k)$ with $d_{\min} = 3$.

Ans:

We need to find $k = 2^m - 1 - m \geq 11$. We use *trial and error* to find the right answer:

- a. Let $m = 1$ $k = 2^m - 1 - m = 2^1 - 1 - 1 = 0$ (not acceptable)
- b. Let $m = 2$ $k = 2^m - 1 - m = 2^2 - 1 - 2 = 1$ (not acceptable)
- c. Let $m = 3$ $k = 2^m - 1 - m = 2^3 - 1 - 3 = 4$ (not acceptable)
- d. Let $m = 4$ $k = 2^m - 1 - m = 2^4 - 1 - 4 = 11$ (acceptable)

Comment: The code is **$C(15, 11)$** with **$d_{\min} = 3$** .

Q.

Which of the following CRC generators guarantee the detection of a single bit error?

- a. $x^3 + x + 1$
- b. $x^4 + x^2$
- c. 1
- d. $x^2 + 1$

Note:

If $g(x)$ has at least two terms (which is normally the case) and the coefficient of x^0 is not zero (the rightmost bit is 1), then error polynomial $e(x)$ cannot be divided by $g(x)$.

Ans:

- a. $x^3 + x + 1 \rightarrow$ It meets both critic.
- b. $x^4 + x^2 \rightarrow$ It meets the first criteria, but not the second.
- c. $1 \rightarrow$ It meets the second criteria, but not the first.
- d. $x^2 + 1 \rightarrow$ It meets both criteria.

Q. Consider the (31,15) Reed–Solomon code.

- a. How many bits are there in a symbol of the code?
- b. What is the block length in bits?
- c. What is the minimum distance of the code?
- d. How many symbols in error can the code correct?

Ans:

a. 5 - bits

b. $31 \times 5 \text{ bits} = 155 \text{ bits}$

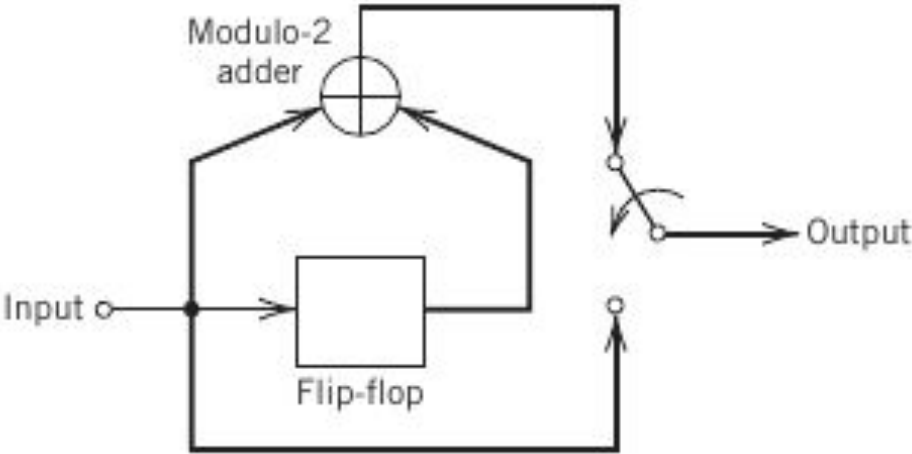
c. $d_{\min} = 2t + 1 = n - k + 1 = 17 \text{ symbols}$

d. 8 symbols

A t -error-correcting Reed–Solomon code has the following parameters:

block length	$n = 2^m - 1$ symbols
message size	k symbols
parity-check size	$n - k = 2t$ symbols
minimum distance	$d_{\min} = 2t + 1$ symbols

Q. Consider a systematic convolutional encoder as shown below. Find the encoder output produced by the message sequence 10111.



Ans: 11 10 11 01 01 01 ..

References:

1. A Practical Guide to Error-Control Coding using MATLAB, Yuan Jiang
2. Error Control Coding, Shu Lin
3. Factors of polynomial in GF(2):

<https://www.ece.unb.ca/tervo/ece4253/polytable.shtml>

4. Irreducible factors of $X^N + 1$ in GF(2), where $N = 2^{(\text{degree of the polynomial})} - 1$

<https://www.ece.unb.ca/tervo/ece4253/factors.shtml>